```
# run_gate_report.py

from __future__ import annotations

import argparse
import math
import multiprocessing as mp
import os
import random
import time
from typing import Dict, Any, List, Tuple, Optional

import numpy as np

from src.data_structures import Layout
from src.physics import PhysicalParameters, get_potential
from src.gates import (
    get_3input_gate,
    get_all_3input_gate_names,
    normalize_gate_name,
)
from src.paper_benchmarks import (
    paper_like_skeleton_3in_1out_25,
    paper_like_canvas_143,
)
from src.filters import (
 F1_min_distance,
 F2_symmetry,
 F3_wire_interference,
 F4_positive_charge,
 F5_charge_count_bound,
```

```
 F6_input_pin_disturbance,
 F7_path_connectivity,
 F8_output_potential_bound,
 F9_electrostatic_pressure,
 F10_energy_lower_bound,
 F11_physical_infeasibility,
)
from src.reporting import (
    write_csv,
    write_json,
    write_markdown,
    markdown_table,
)



QC12_FILTERS = [
    F1_min_distance,
    F2_symmetry,
    F3_wire_interference,
    F4_positive_charge,
    F5_charge_count_bound,
    F6_input_pin_disturbance,
    F7_path_connectivity,
    F8_output_potential_bound,
    F9_electrostatic_pressure,
    F10_energy_lower_bound,
]



_G: Dict[str, Any] = {}
```

```
def _override(default_value, override_value):
    return default_value if override_value is None else override_value




def make_params(
    profile: str,
    io_margin_override: Optional[float],
    pressure_margin_override: Optional[float],
    wire_radius_override: Optional[float],
    input_disturbance_override: Optional[float],
    connectivity_radius_override: Optional[float] = None,
) -> PhysicalParameters:
    profile = str(profile).strip().lower()

    if profile == "safe":
        return PhysicalParameters(
            mu_minus=-0.31,
            mu_plus=-0.80,
            lambda_tf=5.0,
            epsilon_r=5.6,
            check_fixed_io_population=False,
            check_configuration_stability=False,
            relaxed_enumeration=True,
            instability_check_inputs=False,
            f10_check_input_inversions=False,
            charge_min_fraction=0.0,
            charge_max_fraction=1.0,
            bit0_requires_positive_pressure=False,
            wire_forbidden_radius=_override(0.35, wire_radius_override),
```

```
        connectivity_radius=connectivity_radius_override,
        pressure_margin=_override(0.0, pressure_margin_override),
        energy_bound_margin=None,
        io_instability_margin=_override(0.02, io_margin_override),
        input_disturbance_limit=input_disturbance_override,
    )

if profile == "balanced":
    return PhysicalParameters(
        mu_minus=-0.31,
        mu_plus=-0.80,
        lambda_tf=5.0,
        epsilon_r=5.6,
        check_fixed_io_population=False,
        check_configuration_stability=False,
        relaxed_enumeration=True,
        instability_check_inputs=False,
        f10_check_input_inversions=False,
        charge_min_fraction=0.0,
        charge_max_fraction=1.0,
        bit0_requires_positive_pressure=False,
        wire_forbidden_radius=_override(0.40, wire_radius_override),
        connectivity_radius=connectivity_radius_override,
        pressure_margin=_override(0.0, pressure_margin_override),
        energy_bound_margin=None,
        io_instability_margin=_override(0.02, io_margin_override),
        input_disturbance_limit=input_disturbance_override,
    )

if profile == "strict":
    return PhysicalParameters(
```

```
        mu_minus=-0.31,
        mu_plus=-0.80,
        lambda_tf=5.0,
        epsilon_r=5.6,
        check_fixed_io_population=False,
        check_configuration_stability=False,
        relaxed_enumeration=True,
        instability_check_inputs=False,
        f10_check_input_inversions=False,
        charge_min_fraction=0.0,
        charge_max_fraction=1.0,
        bit0_requires_positive_pressure=False,
        wire_forbidden_radius=_override(0.45, wire_radius_override),
        connectivity_radius=_override(2.5 * 5.0, connectivity_radius_override),
        pressure_margin=_override(0.0, pressure_margin_override),
        energy_bound_margin=None,
        io_instability_margin=_override(0.02, io_margin_override),
        input_disturbance_limit=_override(0.45, input_disturbance_override),
    )

  raise ValueError("Unknown strict profile. Use: safe, balanced, strict")



def fixed_charges_ordered(skeleton, x: Tuple[int, ...], y: Tuple[int, ...]) -> List[int]:
  charges = []

  for i, pin in enumerate(skeleton.input_pins):
    charges.extend([q for _, q in pin.charges_for_bit(x[i])])

  for i, pin in enumerate(skeleton.output_pins):
    charges.extend([q for _, q in pin.charges_for_bit(y[i])])
```

```
    for _ in skeleton.wire_sidbs:
        charges.append(0)

    return charges



def precompute_potential_matrix(skeleton, canvas, params):
    positions = skeleton.all_positions() + canvas.positions
    n = len(positions)

    P = np.zeros((n, n), dtype=float)

    for i in range(n):
        for j in range(i + 1, n):
            v = get_potential(positions[i], positions[j], params)
            P[i, j] = v
            P[j, i] = v

    return P



def build_fixed_data(skeleton, func, params, P):
    n_s = len(skeleton.all_sidbs)
    data = {}

    for x in func.all_inputs():
        for y in func.all_outputs():
            charges = fixed_charges_ordered(skeleton, x, y)

            e_fixed_pair = 0.0
```

```
        for i in range(n_s):
            for j in range(i + 1, n_s):
                e_fixed_pair += charges[i] * charges[j] * P[i, j]

        n_neg = sum(1 for q in charges if q == -1)
        e_fixed = e_fixed_pair + n_neg * params.mu_minus * abs(params.qe)

        data[(tuple(x), tuple(y))] = {
            "charges": charges,
            "e_fixed": e_fixed,
        }

    return data


def _init_worker(skeleton, canvas, params, func):
    _G["skeleton"] = skeleton
    _G["canvas"] = canvas
    _G["params"] = params
    _G["func"] = func

    P = precompute_potential_matrix(skeleton, canvas, params)
    _G["P"] = P
    _G["fixed_data"] = build_fixed_data(skeleton, func, params, P)
    _G["n_skel"] = len(skeleton.all_sidbs)


def make_layout(combo: Tuple[int, ...]) -> Layout:
    canvas = _G["canvas"]
    skeleton = _G["skeleton"]
```

```
    positions = [canvas.positions[i] for i in combo]

    return Layout(
        skeleton=skeleton,
        canvas_positions=positions,
        canvas_indices=combo,
    )



def min_relaxed_energy(combo: Tuple[int, ...], x: Tuple[int, ...], y: Tuple[int, ...]) -> float:
    params = _G["params"]
    P = _G["P"]
    n_s = _G["n_skel"]
    fixed_data = _G["fixed_data"][(tuple(x), tuple(y))]

    fixed_charges = fixed_data["charges"]
    e_fixed = fixed_data["e_fixed"]

    canvas_global = [n_s + i for i in combo]
    d = len(canvas_global)

    linear = []

    for cidx in canvas_global:
        contribution = params.mu_minus * abs(params.qe)

        for i, q in enumerate(fixed_charges):
            contribution += q * (-1) * P[i, cidx]

        linear.append(contribution)
```

```
    pair_cc = np.zeros((d, d), dtype=float)

    for i in range(d):
        for j in range(i + 1, d):
            pair_cc[i, j] = P[canvas_global[i], canvas_global[j]]

    best = float("inf")

    for mask in range(1 << d):
        e = e_fixed
        active = []

        for i in range(d):
            if mask & (1 << i):
                e += linear[i]
                active.append(i)

        for a_i in range(len(active)):
            for b_i in range(a_i + 1, len(active)):
                e += pair_cc[active[a_i], active[b_i]]

        if e < best:
            best = e

    return best



def fast_f12_relaxed(combo: Tuple[int, ...]) -> bool:
    params = _G["params"]
    func = _G["func"]
```

```
    tol = max(params.energy_tolerance, params.io_instability_margin)

    for x in func.all_inputs():
        y_correct = func.eval(x)
        e_correct = min_relaxed_energy(combo, tuple(x), tuple(y_correct))

        for y_alt in func.all_outputs():
            y_alt = tuple(y_alt)

            if y_alt == tuple(y_correct):
                continue

            e_alt = min_relaxed_energy(combo, tuple(x), y_alt)

            if e_alt < e_correct - tol:
                return True

    return False



def fast_energy_margin(combo: Tuple[int, ...]) -> float:
    func = _G["func"]
    min_margin = float("inf")

    for x in func.all_inputs():
        y_correct = tuple(func.eval(x))
        e_correct = min_relaxed_energy(combo, tuple(x), y_correct)

        best_alt = float("inf")
```

```
        for y_alt in func.all_outputs():
            y_alt = tuple(y_alt)

            if y_alt == y_correct:
                continue

            e_alt = min_relaxed_energy(combo, tuple(x), y_alt)

            if e_alt < best_alt:
                best_alt = e_alt

        margin = best_alt - e_correct

        if margin < min_margin:
            min_margin = margin

    return float(min_margin)



def run_filter_sequence_for_qc12(layout: Layout) -> Tuple[List[int], bool]:
    params = _G["params"]
    func = _G["func"]
    skeleton = _G["skeleton"]
    canvas = _G["canvas"]

    after = [0] * 12

    for i, f in enumerate(QC12_FILTERS):
        discard = f(
            layout=layout,
            params=params,
```

```
        func=func,
        skeleton=skeleton,
        canvas=canvas,
    )

    if discard:
        return after, False

    after[i] = 1

# F11: Physical infeasibility pruning.
# Modes:
#   counted    : do not execute F11, only count as survived
#   strict-pop : execute F11 with population stability only
#   strict-hop : execute F11 with population + hop stability
f11_mode = getattr(params, "f11_mode", "counted")

if f11_mode == "counted":
    after[10] = 1
else:
    discard_f11 = F11_physical_infeasibility(
        layout=layout,
        params=params,
        func=func,
        skeleton=skeleton,
        canvas=canvas,
    )

    if discard_f11:
        return after, False
```

```
        after[10] = 1


    discard_f12 = fast_f12_relaxed(tuple(layout.canvas_indices))


    if discard_f12:
        return after, False


    after[11] = 1


    return after, True



def run_filter_sequence_for_qc3(layout: Layout) -> Dict[str, int]:
    params = _G["params"]
    func = _G["func"]
    skeleton = _G["skeleton"]
    canvas = _G["canvas"]


    out = {
        "after_f4": 0,
        "after_f11": 0,
        "after_f12": 0,
    }


    discard_f4 = F4_positive_charge(
        layout=layout,
        params=params,
        func=func,
        skeleton=skeleton,
        canvas=canvas,
    )
```

```
    if discard_f4:
        return out

    out["after_f4"] = 1
    out["after_f11"] = 1

    discard_f12 = fast_f12_relaxed(tuple(layout.canvas_indices))

    if discard_f12:
        return out

    out["after_f12"] = 1

    return out



def worker_process_chunk(chunk: List[Tuple[int, ...]]) -> Dict[str, Any]:
    qc12_after = [0] * 12
    qc12_passed = []

    qc3_after_f4 = 0
    qc3_after_f11 = 0
    qc3_after_f12 = 0

    for combo in chunk:
        layout = make_layout(combo)

        after, passed = run_filter_sequence_for_qc12(layout)

        for i in range(12):
```

```
            qc12_after[i] += after[i]

        if passed:
            qc12_passed.append(combo)

        qc3 = run_filter_sequence_for_qc3(layout)

        qc3_after_f4 += qc3["after_f4"]
        qc3_after_f11 += qc3["after_f11"]
        qc3_after_f12 += qc3["after_f12"]

    return {
        "processed": len(chunk),
        "qc12_after": qc12_after,
        "qc12_passed": qc12_passed,
        "qc3_after_f4": qc3_after_f4,
        "qc3_after_f11": qc3_after_f11,
        "qc3_after_f12": qc3_after_f12,
    }


def random_combinations(n: int, d: int, count: int, seed: int):
    rng = random.Random(seed)
    seen = set()
    total_possible = math.comb(n, d)
    count = min(count, total_possible)

    while len(seen) < count:
        combo = tuple(sorted(rng.sample(range(n), d)))

        if combo in seen:
```

```
            continue

        seen.add(combo)
        yield combo


def chunked(iterable, chunk_size: int):
    batch = []

    for item in iterable:
        batch.append(item)

        if len(batch) >= chunk_size:
            yield batch
            batch = []

    if batch:
        yield batch


def merge(total: Dict[str, Any], part: Dict[str, Any]):
    total["processed"] += part["processed"]

    for i in range(12):
        total["qc12_after"][i] += part["qc12_after"][i]

    total["qc12_passed"].extend(part["qc12_passed"])

    total["qc3_after_f4"] += part["qc3_after_f4"]
    total["qc3_after_f11"] += part["qc3_after_f11"]
    total["qc3_after_f12"] += part["qc3_after_f12"]
```

```
def select_ranked_candidates(
    combos: List[Tuple[int, ...]],
    target: Optional[int],
    min_margin: Optional[float],
) -> Tuple[List[Tuple[int, ...]], List[Dict[str, Any]]]:
    scored = []

    for combo in combos:
        margin = fast_energy_margin(combo)
        scored.append((margin, combo))

    scored.sort(key=lambda t: (-t[0], t[1]))

    selected = []
    rows = []

    for margin, combo in scored:
        if min_margin is not None and margin < min_margin:
            continue

        if target is not None and target > 0 and len(selected) >= target:
            break

        selected.append(combo)

        rows.append({
            "rank": len(selected),
            "energy_margin": margin,
            "canvas_indices": " ".join(str(i) for i in combo),
```

```
        })

    return selected, rows



def compute_phase_rows(total: Dict[str, Any], elapsed: float, selection_enabled: bool):
    processed = total["processed"]
    a = total["qc12_after"]

    rows = []

    def row(name, filters, inp, out, runtime=""):
        retention = out / inp if inp else 0.0
        pruning = 1.0 - retention
        reduction = inp / out if out else float("inf")

        return {
            "Phase": name,
            "Filters": filters,
            "Input Layouts": inp,
            "Output Layouts": out,
            "Retention Ratio [%]": round(retention * 100.0, 4),
            "Pruning Ratio [%]": round(pruning * 100.0, 4),
            "Reduction Factor": "inf" if reduction == float("inf") else round(reduction, 4),
            "Runtime [s]": runtime,
        }

    rows.append(row("Phase 1", "F1-F4", processed, a[3]))
    rows.append(row("Phase 2", "F5-F10", a[3], a[9]))
    rows.append(row("Phase 3", "F11-F12", a[9], a[11]))
```

```
    if selection_enabled:
        rows.append(row("Post-F12 Selection", "Energy-margin ranking", a[11], total["selected_candidates"]))
        final_out = total["selected_candidates"]
        total_label = "F1-F12 + selection"
    else:
        final_out = a[11]
        total_label = "F1-F12"

    rows.append(row("Total", total_label, processed, final_out, round(elapsed, 4)))

    return rows



def compute_comparison_rows(total: Dict[str, Any], selection_enabled: bool):
    processed = total["processed"]
    a = total["qc12_after"]

    qc3_before = total["qc3_after_f12"]
    qc12_before = total["selected_candidates"] if selection_enabled else a[11]

    qc3_after_early = total["qc3_after_f4"]
    qc12_after_phase1 = a[3]

    qc12_before_enum = a[9]
    qc3_before_enum = qc3_after_early

    def improvement_fewer(old, new):
        if new == 0:
            return "inf"
        return f"{old / new:.2f}x fewer"
```

```
return [
  {
    "Metric": "Number of pruning filters",
    "Original QC-3": 3,
    "QuickCell-12": 12,
    "Improvement": "4.0x more",
  },
  {
    "Metric": "Processed layouts",
    "Original QC-3": processed,
    "QuickCell-12": processed,
    "Improvement": "same",
  },
  {
    "Metric": "After early pruning / Phase 1",
    "Original QC-3": qc3_after_early,
    "QuickCell-12": qc12_after_phase1,
    "Improvement": improvement_fewer(qc3_after_early, qc12_after_phase1),
  },
  {
    "Metric": "Before expensive enumerative pruning",
    "Original QC-3": qc3_before_enum,
    "QuickCell-12": qc12_before_enum,
    "Improvement": improvement_fewer(qc3_before_enum, qc12_before_enum),
  },
  {
    "Metric": "Before final verification",
    "Original QC-3": qc3_before,
    "QuickCell-12": qc12_before,
    "Improvement": improvement_fewer(qc3_before, qc12_before),
  },
```

```
    {
        "Metric": "Candidate implementations",
        "Original QC-3": qc3_before,
        "QuickCell-12": qc12_before,
        "Improvement": improvement_fewer(qc3_before, qc12_before),
    },
    {
        "Metric": "Search-space reduction",
        "Original QC-3": round(processed / qc3_before, 4) if qc3_before else "inf",
        "QuickCell-12": round(processed / qc12_before, 4) if qc12_before else "inf",
        "Improvement": improvement_fewer(qc3_before, qc12_before),
    },
]



def compute_speedup_rows(total: Dict[str, Any], full_ld: int, elapsed: float, sim_time: float, selection_enabled: bool):
    processed = total["processed"]

    qc3_candidates = total["qc3_after_f12"]
    qc_candidates = total["selected_candidates"] if selection_enabled else total["qc12_after"][11]

    limited_sota = processed * 8 * sim_time
    full_sota = full_ld * 8 * sim_time

    return [
        {"Quantity": "Processed layouts", "Value": processed},
        {"Quantity": "Full search space |Ld|", "Value": full_ld},
        {"Quantity": "Input patterns for 3-input gate", "Value": 8},
        {"Quantity": "Assumed simulation time per layout/input [s]", "Value": sim_time},
        {"Quantity": "Estimated brute-force time for processed sample [s]", "Value": round(limited_sota, 4)},
        {"Quantity": "Estimated brute-force time for full |Ld| [s]", "Value": round(full_sota, 4)},
```

```
        {"Quantity": "QC-3 candidates before verification", "Value": qc3_candidates},
        {"Quantity": "QuickCell-12 candidates before verification", "Value": qc_candidates},
        {
            "Quantity": "QuickCell-12 candidate reduction over QC-3",
            "Value": f"{(qc3_candidates / qc_candidates):.4f}x" if qc_candidates else "inf",
        },
        {"Quantity": "Measured QuickCell-12 wall time [s]", "Value": round(elapsed, 4)},
    ]



def write_reports(
    gate,
    full_ld,
    total,
    elapsed,
    output_dir,
    sim_time,
    selection_enabled,
    selected_rows,
):
    os.makedirs(output_dir, exist_ok=True)

    a = total["qc12_after"]

    table_i = [{
        "Benchmark": gate,
        "|Ld|": full_ld,
        "Processed": total["processed"],
        "After F1": a[0],
        "After F2": a[1],
        "After F3": a[2],
```

```
    "After F4": a[3],
    "After F5": a[4],
    "After F6": a[5],
    "After F7": a[6],
    "After F8": a[7],
    "After F9": a[8],
    "After F10": a[9],
    "After F11": a[10],
    "After F12": a[11],
    "Selected Candidates": total["selected_candidates"] if selection_enabled else "",
    "Candidates": total["selected_candidates"] if selection_enabled else a[11],
    "Total Time [s]": round(elapsed, 4),
}]

table_ii = compute_phase_rows(total, elapsed, selection_enabled)
table_iii = compute_comparison_rows(total, selection_enabled)
speedup = compute_speedup_rows(total, full_ld, elapsed, sim_time, selection_enabled)

write_csv(os.path.join(output_dir, "table_I_ablation.csv"), table_i, list(table_i[0].keys()))
write_csv(os.path.join(output_dir, "table_II_phase_summary.csv"), table_ii, list(table_ii[0].keys()))
write_csv(os.path.join(output_dir, "table_III_comparison.csv"), table_iii, list(table_iii[0].keys()))
write_csv(os.path.join(output_dir, "speedup_calculation.csv"), speedup, list(speedup[0].keys()))

if selected_rows:
    write_csv(
        os.path.join(output_dir, "selected_candidates.csv"),
        selected_rows,
        list(selected_rows[0].keys()),
    )

write_json(
```

```
    os.path.join(output_dir, "report.json"),
    {
        "gate": gate,
        "full_ld": full_ld,
        "total": total,
        "elapsed": elapsed,
        "selection_enabled": selection_enabled,
        "table_I": table_i,
        "table_II": table_ii,
        "table_III": table_iii,
        "speedup": speedup,
        "selected_candidates": selected_rows,
    },
)

md = []

md.append(f"# QuickCell-12 Limited-run Report: {gate}\n")
md.append("## Table I — Ablation Results\n")
md.append(markdown_table(list(table_i[0].keys()), [list(table_i[0].values())]))
md.append("\n\n## Table II — Phase-wise Summary\n")
md.append(markdown_table(list(table_ii[0].keys()), [list(r.values()) for r in table_ii]))
md.append("\n\n## Table III — Original QC-3 vs QuickCell-12\n")
md.append(markdown_table(list(table_iii[0].keys()), [list(r.values()) for r in table_iii]))
md.append("\n\n## Numerical Speedup Calculation\n")
md.append(markdown_table(list(speedup[0].keys()), [list(r.values()) for r in speedup]))
md.append("\n\n**Note:** This is a limited-run / sampling-based evaluation.\n")

write_markdown(os.path.join(output_dir, "report.md"), "\n".join(md))
```

```
def parse_gate_argument(gate_arg: str):
    gate_arg = gate_arg.strip()

    if gate_arg.upper() == "ALL":
        return get_all_3input_gate_names()

    return [normalize_gate_name(g) for g in gate_arg.split(",") if g.strip()]



def run_gate(
    gate,
    samples,
    workers,
    chunksize,
    seed,
    sim_time,
    strict_profile,
    io_margin,
    pressure_margin,
    connectivity_radius,
    d_min,
    positive_charge_margin,
    charge_min_fraction,
    charge_max_fraction,
    charge_count_conflict_radius,
    input_disturbance,
    energy_margin,
    f11_mode,
    f11_min_support_states,
    selection_target,
    selection_min_margin,
```

```
):

    gate = normalize_gate_name(gate)

    func = get_3input_gate(gate)
    skeleton = paper_like_skeleton_3in_1out_25()
    canvas = paper_like_canvas_143()

    params = make_params(
        profile=strict_profile,
        io_margin_override=io_margin,
        pressure_margin_override=pressure_margin,
        wire_radius_override=None,
        input_disturbance_override=input_disturbance,
        connectivity_radius_override=connectivity_radius,
    )

    params.f11_mode = f11_mode
    if f11_mode == "counted":
        # Old behavior: F11 is only counted as survived.
        params.relaxed_enumeration = True
        params.check_configuration_stability = False

    elif f11_mode == "strict-pop":
        # Real F11, but only population stability is checked.
        # Less destructive than strict-hop.
        params.relaxed_enumeration = False
        params.check_configuration_stability = False
        params.check_fixed_io_population = False

    elif f11_mode == "strict-hop":
```

```
    # Real F11 with population stability and single-electron hop stability.
    # This can be very strict.
    params.relaxed_enumeration = False
    params.check_configuration_stability = True
    params.check_fixed_io_population = False

if d_min is not None:
    params.d_min = float(d_min)
params.positive_charge_margin = float(positive_charge_margin or 0.0)
if charge_min_fraction is not None:
    params.charge_min_fraction = float(charge_min_fraction)
if charge_max_fraction is not None:
    params.charge_max_fraction = float(charge_max_fraction)
params.charge_count_conflict_radius = charge_count_conflict_radius
params.energy_bound_margin = energy_margin
params.f11_min_support_states = int(f11_min_support_states)

full_ld = math.comb(canvas.n_pos, 4)

selection_enabled = (
    selection_target is not None and selection_target > 0
) or (
    selection_min_margin is not None
)

print("=" * 80)
print(f"Gate: {gate}")
print(f"|C| = {canvas.n_pos}, d = 4, |Ld| = {full_ld:,}")
print(f"Processed samples = {samples:,}")
print(f"Workers = {workers}, chunksize = {chunksize}")
print(f"Strict profile = {strict_profile}")
```

```
print(f"F12 I/O margin = {params.io_instability_margin}")
print(f"F9 pressure margin = {params.pressure_margin}")
print(f"F7 connectivity radius = {params.connectivity_radius}")
print(f"F1 minimum distance d_min = {params.d_min}")
print(f"F4 positive charge margin = {params.positive_charge_margin}")
print(f"F5 charge fraction = [{params.charge_min_fraction}, {params.charge_max_fraction}]")
print(f"F5 charge-count conflict radius = {params.charge_count_conflict_radius}")
print(f"F6 input disturbance limit = {params.input_disturbance_limit}")
print(f"F10 energy margin = {params.energy_bound_margin}")
print(f"F11 mode = {params.f11_mode}")
print(f"F11 relaxed enumeration = {params.relaxed_enumeration}")

print(f"F11 mode = {params.f11_mode}")
print(f"F11 relaxed enumeration = {params.relaxed_enumeration}")
print(f"F11 hop stability check = {params.check_configuration_stability}")
print(f"F11 check fixed I/O population = {params.check_fixed_io_population}")
print(f"F11 min support states = {params.f11_min_support_states}")



print(f"F11 hop stability check = {params.check_configuration_stability}")
print(f"Post-F12 selection enabled = {selection_enabled}")
print(f"Selection target = {selection_target}")
print(f"Selection min margin = {selection_min_margin}")
print("=" * 80)

total = {
    "processed": 0,
    "qc12_after": [0] * 12,
    "qc12_passed": [],
    "qc3_after_f4": 0,
    "qc3_after_f11": 0,
```

```
    "qc3_after_f12": 0,
    "selected_candidates": 0,
}

combos = random_combinations(canvas.n_pos, 4, samples, seed)
chunks = list(chunked(combos, chunksize))

start = time.time()

if workers <= 1:
    _init_worker(skeleton, canvas, params, func)

    for chunk in chunks:
        part = worker_process_chunk(chunk)
        merge(total, part)
else:
    ctx = mp.get_context("spawn")

    with ctx.Pool(
        processes=workers,
        initializer=_init_worker,
        initargs=(skeleton, canvas, params, func),
    ) as pool:
        for idx, part in enumerate(pool.imap_unordered(worker_process_chunk, chunks), 1):
            merge(total, part)

            if idx % 5 == 0 or idx == len(chunks):
                print(
                    f"  progress: {total['processed']:,}/{samples:,} "
                    f"({100.0 * total['processed'] / samples:.2f}%)"
                )
```

```
# Re-initialize in main process for ranking.
_init_worker(skeleton, canvas, params, func)

selected_rows = []

if selection_enabled:
    selected_combos, selected_rows = select_ranked_candidates(
        total["qc12_passed"],
        target=selection_target,
        min_margin=selection_min_margin,
    )
    total["selected_candidates"] = len(selected_combos)
else:
    total["selected_candidates"] = total["qc12_after"][11]

elapsed = time.time() - start

output_dir = os.path.join("results", gate)

write_reports(
    gate=gate,
    full_ld=full_ld,
    total=total,
    elapsed=elapsed,
    output_dir=output_dir,
    sim_time=sim_time,
    selection_enabled=selection_enabled,
    selected_rows=selected_rows,
)
```

```
    print("\nResult summary:")
    print(f"  Processed: {total['processed']:,}")
    print(f"  After F12: {total['qc12_after'][11]:,}")
    if selection_enabled:
        print(f"  Selected candidates: {total['selected_candidates']:,}")
    print(f"  Final Candidates: {total['selected_candidates']:,}")
    print(f"  QC-3 After F12: {total['qc3_after_f12']:,}")

    if total["selected_candidates"]:
        print(f"  Reduction over QC-3: {total['qc3_after_f12'] / total['selected_candidates']:.2f}x")

    print(f"  Wall time: {elapsed:.2f} s")
    print(f"  Output directory: {output_dir}")


def main():
    parser = argparse.ArgumentParser(
        description="Run limited sample report for QuickCell-12 vs QC-3."
    )
    parser.add_argument(
    "--connectivity-radius",
    type=float,
    default=None,
    help="Override F7 connectivity radius. Smaller is stricter.",
    )

    parser.add_argument(
    "--d-min",
    type=float,
    default=None,
    help="Override F1 minimum canvas-canvas distance. Larger is stricter.",
```

```
)

parser.add_argument(
"--positive-charge-margin",
type=float,
default=0.0,
help="Override F4 positive-charge safety margin. Larger is stricter.",
)

parser.add_argument(
"--charge-min-fraction",
type=float,
default=None,
help="Override F5 minimum negative-charge fraction.",
)

parser.add_argument(
"--charge-max-fraction",
type=float,
default=None,
help="Override F5 maximum negative-charge fraction.",
)

parser.add_argument(
"--charge-count-conflict-radius",
type=float,
default=None,
help="F5 canvas negative-charge conflict radius. Larger is stricter.",
)

parser.add_argument(
```

```
"--input-disturbance",
type=float,
default=None,
help="Override F6 input disturbance limit. Smaller is stricter.",
)

parser.add_argument(
"--energy-margin",
type=float,
default=None,
help="Override F10 relaxed energy margin. Smaller is stricter.",
)



parser.add_argument(
    "--f11-mode",
    default="counted",
    choices=["counted", "strict-pop", "strict-hop"],
    help=(
        "F11 execution mode. "
        "counted = do not run F11; "
        "strict-pop = population stability only; "
        "strict-hop = population + hop stability."
    ),
)

parser.add_argument(
"--f11-min-support-states",
type=int,
default=1,
help="F11 minimum number of metastable supporting canvas states per input. Larger is stricter.",
```

```
)

parser.add_argument("--gate", required=True, help="Gate name, comma-separated list, or ALL")
parser.add_argument("--samples", type=int, default=100000)
parser.add_argument("--workers", type=int, default=4)
parser.add_argument("--chunksize", type=int, default=2000)
parser.add_argument("--seed", type=int, default=12345)
parser.add_argument("--sim-time", type=float, default=0.01)

parser.add_argument(
    "--strict-profile",
    default="balanced",
    choices=["safe", "balanced", "strict"],
)

parser.add_argument(
    "--io-margin",
    type=float,
    default=None,
    help="Override F12 I/O instability margin. Smaller is stricter.",
)

parser.add_argument(
    "--pressure-margin",
    type=float,
    default=None,
    help="Override F9 pressure margin. Positive values may be too strict.",
)

parser.add_argument(
    "--selection-target",
```

```
    type=int,
    default=0,
    help="Optional post-F12 top-k selection target. 0 disables.",
)

parser.add_argument(
    "--selection-min-margin",
    type=float,
    default=None,
    help="Optional post-F12 minimum energy margin.",
)

args = parser.parse_args()

gates = parse_gate_argument(args.gate)

selection_target = args.selection_target if args.selection_target > 0 else None

for gate in gates:
    run_gate(
        gate=gate,
        samples=args.samples,
        workers=args.workers,
        chunksize=args.chunksize,
        seed=args.seed,
        sim_time=args.sim_time,
        strict_profile=args.strict_profile,
        io_margin=args.io_margin,
        pressure_margin=args.pressure_margin,
        connectivity_radius=args.connectivity_radius,
        d_min=args.d_min,
```

```
        positive_charge_margin=args.positive_charge_margin,
        charge_min_fraction=args.charge_min_fraction,
        charge_max_fraction=args.charge_max_fraction,
        charge_count_conflict_radius=args.charge_count_conflict_radius,
        input_disturbance=args.input_disturbance,
        energy_margin=args.energy_margin,
        f11_mode=args.f11_mode,
        f11_min_support_states=args.f11_min_support_states,
        selection_target=selection_target,
        selection_min_margin=args.selection_min_margin,
    )



if __name__ == "__main__":
    main()
```

---

```
# main.py


print("QuickCell-12 utilities")
print()
print("Generate tables only:")
print("python run_gate_report.py --gate AND3 --samples 100000 --workers 4 --chunksize 2000 --strict-profile balanced --io-margin 0.0001")
print()
print("Generate tables + SiQAD validation files:")
print("python export_siqad_validation.py --gate AND3 --samples 100000 --workers 4 --chunksize 2000 --strict-profile balanced --io-margin 0.0001 --max-candidates 50")
print()
print("Generate for all gates:")
print("python export_siqad_validation.py --gate ALL --samples 100000 --workers 4 --chunksize 2000 --strict-profile balanced --io-margin 0.0001 --max-candidates 50")
```

```
print()
print("Analyze filled SiQAD validation sheets:")
print("python analyze_siqad_results.py --root validation_exports --all")
```

---

```
# export_siqad_validation.py
from __future__ import annotations

import argparse
import csv
import json
import math
import multiprocessing as mp
import os
import random
import time
from typing import Dict, Any, List, Tuple, Optional

import numpy as np

from src.data_structures import Layout
from src.physics import PhysicalParameters, get_potential
from src.gates import (
    get_3input_gate,
    get_all_3input_gate_names,
    normalize_gate_name,
)
from src.paper_benchmarks import (
    paper_like_skeleton_3in_1out_25,
    paper_like_canvas_143,
)
from src.filters import (
```

```
    F1_min_distance,
    F2_symmetry,
    F3_wire_interference,
    F4_positive_charge,
    F5_charge_count_bound,
    F6_input_pin_disturbance,
    F7_path_connectivity,
    F8_output_potential_bound,
    F9_electrostatic_pressure,
    F10_energy_lower_bound,
    F11_physical_infeasibility,
)
from src.siqad_export import write_candidate_export



QC12_FILTERS = [
    F1_min_distance,
    F2_symmetry,
    F3_wire_interference,
    F4_positive_charge,
    F5_charge_count_bound,
    F6_input_pin_disturbance,
    F7_path_connectivity,
    F8_output_potential_bound,
    F9_electrostatic_pressure,
    F10_energy_lower_bound,
]



_G: Dict[str, Any] = {}
```

```
def _override(default_value, override_value):
    return default_value if override_value is None else override_value



def make_params(
    profile: str,
    io_margin_override: Optional[float],
    pressure_margin_override: Optional[float],
    wire_radius_override: Optional[float],
    input_disturbance_override: Optional[float],
    connectivity_radius_override: Optional[float],
    charge_min_fraction_override: Optional[float],
    charge_max_fraction_override: Optional[float],
    energy_margin_override: Optional[float],
) -> PhysicalParameters:
    """
    Create PhysicalParameters consistent with run_gate_report.py.

    Important:
    Some additional parameters such as d_min, positive_charge_margin,
    charge_count_conflict_radius, f11_mode, and f11_min_support_states
    are attached after this function, because they are either derived
    attributes or prototype-specific tuning attributes.
    """
    profile = str(profile).strip().lower()

    if profile == "safe":
        return PhysicalParameters(
            mu_minus=-0.31,
            mu_plus=-0.80,
```

```
        lambda_tf=5.0,
        epsilon_r=5.6,
        check_fixed_io_population=False,
        check_configuration_stability=False,
        relaxed_enumeration=True,
        instability_check_inputs=False,
        f10_check_input_inversions=False,
        charge_min_fraction=_override(0.0, charge_min_fraction_override),
        charge_max_fraction=_override(1.0, charge_max_fraction_override),
        bit0_requires_positive_pressure=False,
        wire_forbidden_radius=_override(0.35, wire_radius_override),
        connectivity_radius=connectivity_radius_override,
        pressure_margin=_override(0.0, pressure_margin_override),
        energy_bound_margin=energy_margin_override,
        io_instability_margin=_override(0.02, io_margin_override),
        input_disturbance_limit=input_disturbance_override,
    )

if profile == "balanced":
    return PhysicalParameters(
        mu_minus=-0.31,
        mu_plus=-0.80,
        lambda_tf=5.0,
        epsilon_r=5.6,
        check_fixed_io_population=False,
        check_configuration_stability=False,
        relaxed_enumeration=True,
        instability_check_inputs=False,
        f10_check_input_inversions=False,
        charge_min_fraction=_override(0.0, charge_min_fraction_override),
        charge_max_fraction=_override(1.0, charge_max_fraction_override),
```

```
        bit0_requires_positive_pressure=False,
        wire_forbidden_radius=_override(0.40, wire_radius_override),
        connectivity_radius=connectivity_radius_override,
        pressure_margin=_override(0.0, pressure_margin_override),
        energy_bound_margin=energy_margin_override,
        io_instability_margin=_override(0.02, io_margin_override),
        input_disturbance_limit=input_disturbance_override,
    )

if profile == "strict":
    return PhysicalParameters(
        mu_minus=-0.31,
        mu_plus=-0.80,
        lambda_tf=5.0,
        epsilon_r=5.6,
        check_fixed_io_population=False,
        check_configuration_stability=False,
        relaxed_enumeration=True,
        instability_check_inputs=False,
        f10_check_input_inversions=False,
        charge_min_fraction=_override(0.0, charge_min_fraction_override),
        charge_max_fraction=_override(1.0, charge_max_fraction_override),
        bit0_requires_positive_pressure=False,
        wire_forbidden_radius=_override(0.45, wire_radius_override),
        connectivity_radius=_override(2.5 * 5.0, connectivity_radius_override),
        pressure_margin=_override(0.0, pressure_margin_override),
        energy_bound_margin=energy_margin_override,
        io_instability_margin=_override(0.02, io_margin_override),
        input_disturbance_limit=_override(0.45, input_disturbance_override),
    )
```

```
    raise ValueError("Unknown profile. Use: safe, balanced, strict")



def apply_runtime_overrides(
    params: PhysicalParameters,
    d_min: Optional[float],
    positive_charge_margin: Optional[float],
    charge_count_conflict_radius: Optional[float],
    f11_mode: str,
    f11_min_support_states: int,
) -> PhysicalParameters:
    """
    Apply prototype-specific tuning parameters after PhysicalParameters creation.
    """
    if d_min is not None:
        params.d_min = float(d_min)

    params.positive_charge_margin = float(positive_charge_margin or 0.0)
    params.charge_count_conflict_radius = charge_count_conflict_radius

    params.f11_mode = str(f11_mode)
    params.f11_min_support_states = int(f11_min_support_states)

    if params.f11_mode == "counted":
        # Old behavior: F11 is only counted as survived.
        params.relaxed_enumeration = True
        params.check_configuration_stability = False

    elif params.f11_mode == "strict-pop":
        # Real F11 with population stability only.
        params.relaxed_enumeration = False
```

```
        params.check_configuration_stability = False
        params.check_fixed_io_population = False

    elif params.f11_mode == "strict-hop":
        # Real F11 with population stability and hop stability.
        params.relaxed_enumeration = False
        params.check_configuration_stability = True
        params.check_fixed_io_population = False

    else:
        raise ValueError("Unknown --f11-mode. Use: counted, strict-pop, strict-hop")

    return params



def fixed_charges_ordered(
    skeleton,
    x: Tuple[int, ...],
    y: Tuple[int, ...],
) -> List[int]:
    charges = []

    for i, pin in enumerate(skeleton.input_pins):
        charges.extend([q for _, q in pin.charges_for_bit(x[i])])

    for i, pin in enumerate(skeleton.output_pins):
        charges.extend([q for _, q in pin.charges_for_bit(y[i])])

    for _ in skeleton.wire_sidbs:
        charges.append(0)
```

```
    return charges



def precompute_potential_matrix(skeleton, canvas, params):
    positions = skeleton.all_positions() + canvas.positions
    n = len(positions)

    P = np.zeros((n, n), dtype=float)

    for i in range(n):
        for j in range(i + 1, n):
            v = get_potential(positions[i], positions[j], params)
            P[i, j] = v
            P[j, i] = v

    return P



def build_fixed_data(skeleton, func, params, P):
    n_s = len(skeleton.all_sidbs)
    data = {}

    for x in func.all_inputs():
        for y in func.all_outputs():
            charges = fixed_charges_ordered(skeleton, x, y)

            e_fixed_pair = 0.0
            for i in range(n_s):
                for j in range(i + 1, n_s):
                    e_fixed_pair += charges[i] * charges[j] * P[i, j]
```

```
        n_neg = sum(1 for q in charges if q == -1)
        e_fixed = e_fixed_pair + n_neg * params.mu_minus * abs(params.qe)

        data[(tuple(x), tuple(y))] = {
            "charges": charges,
            "e_fixed": e_fixed,
        }

    return data



def _init_worker(skeleton, canvas, params, func):
    _G["skeleton"] = skeleton
    _G["canvas"] = canvas
    _G["params"] = params
    _G["func"] = func

    P = precompute_potential_matrix(skeleton, canvas, params)

    _G["P"] = P
    _G["fixed_data"] = build_fixed_data(skeleton, func, params, P)
    _G["n_skel"] = len(skeleton.all_sidbs)



def make_layout(combo: Tuple[int, ...]) -> Layout:
    canvas = _G["canvas"]
    skeleton = _G["skeleton"]

    return Layout(
        skeleton=skeleton,
        canvas_positions=[canvas.positions[i] for i in combo],
```

```
        canvas_indices=combo,
    )



def min_relaxed_energy(
    combo: Tuple[int, ...],
    x: Tuple[int, ...],
    y: Tuple[int, ...],
) -> float:
    params = _G["params"]
    P = _G["P"]
    n_s = _G["n_skel"]

    fixed_data = _G["fixed_data"][(tuple(x), tuple(y))]

    fixed_charges = fixed_data["charges"]
    e_fixed = fixed_data["e_fixed"]

    canvas_global = [n_s + i for i in combo]
    d = len(canvas_global)

    linear = []

    for cidx in canvas_global:
        contribution = params.mu_minus * abs(params.qe)

        for i, q in enumerate(fixed_charges):
            contribution += q * (-1) * P[i, cidx]

        linear.append(contribution)
```

```
    pair_cc = np.zeros((d, d), dtype=float)

    for i in range(d):
        for j in range(i + 1, d):
            pair_cc[i, j] = P[canvas_global[i], canvas_global[j]]

    best = float("inf")

    for mask in range(1 << d):
        e = e_fixed
        active = []

        for i in range(d):
            if mask & (1 << i):
                e += linear[i]
                active.append(i)

        for a_i in range(len(active)):
            for b_i in range(a_i + 1, len(active)):
                e += pair_cc[active[a_i], active[b_i]]

        if e < best:
            best = e

    return best



def fast_f12_relaxed(combo: Tuple[int, ...]) -> bool:
    """
    Return True if layout should be discarded by F12.
    """
```

```
params = _G["params"]
func = _G["func"]

tol = max(params.energy_tolerance, params.io_instability_margin)

for x in func.all_inputs():
    y_correct = func.eval(x)

    e_correct = min_relaxed_energy(
        combo,
        tuple(x),
        tuple(y_correct),
    )

    for y_alt in func.all_outputs():
        y_alt = tuple(y_alt)

        if y_alt == tuple(y_correct):
            continue

        e_alt = min_relaxed_energy(
            combo,
            tuple(x),
            y_alt,
        )

        if e_alt < e_correct - tol:
            return True

return False
```

```
def fast_energy_margin(combo: Tuple[int, ...]) -> float:
    """
    Minimum energy margin over all input patterns:
        best incorrect energy - correct energy

    Larger is better.
    """
    func = _G["func"]

    min_margin = float("inf")

    for x in func.all_inputs():
        y_correct = tuple(func.eval(x))

        e_correct = min_relaxed_energy(
            combo,
            tuple(x),
            y_correct,
        )

        best_alt = float("inf")

        for y_alt in func.all_outputs():
            y_alt = tuple(y_alt)

            if y_alt == y_correct:
                continue

            e_alt = min_relaxed_energy(
                combo,
```

```
            tuple(x),
            y_alt,
        )

        if e_alt < best_alt:
            best_alt = e_alt

    margin = best_alt - e_correct

    if margin < min_margin:
        min_margin = margin

return float(min_margin)



def run_filter_sequence_for_qc12(layout: Layout) -> Tuple[List[int], bool]:
    """
    Run F1-F12 for one layout.

    after[i] = 1 means the layout survived filter i+1.
    Return:
        after, passed_all
    """
    params = _G["params"]
    func = _G["func"]
    skeleton = _G["skeleton"]
    canvas = _G["canvas"]

    after = [0] * 12

    # F1-F10
```

```
for i, f in enumerate(QC12_FILTERS):
    discard = f(
        layout=layout,
        params=params,
        func=func,
        skeleton=skeleton,
        canvas=canvas,
    )

    if discard:
        return after, False

    after[i] = 1

# F11
f11_mode = getattr(params, "f11_mode", "counted")

if f11_mode == "counted":
    after[10] = 1
else:
    discard_f11 = F11_physical_infeasibility(
        layout=layout,
        params=params,
        func=func,
        skeleton=skeleton,
        canvas=canvas,
    )

    if discard_f11:
        return after, False
```

```
        after[10] = 1

    # F12
    discard_f12 = fast_f12_relaxed(tuple(layout.canvas_indices))

    if discard_f12:
        return after, False

    after[11] = 1

    return after, True



def worker_process_chunk(chunk: List[Tuple[int, ...]]) -> Dict[str, Any]:
    qc12_after = [0 for _ in range(12)]
    qc12_passed = []

    for combo in chunk:
        layout = make_layout(combo)

        after, passed = run_filter_sequence_for_qc12(layout)

        for i in range(12):
            qc12_after[i] += after[i]

        if passed:
            qc12_passed.append(combo)

    return {
        "processed": len(chunk),
        "qc12_after": qc12_after,
```

```
        "qc12_passed": qc12_passed,
    }



def random_combinations(n: int, d: int, count: int, seed: int):
    rng = random.Random(seed)

    seen = set()
    total_possible = math.comb(n, d)
    count = min(count, total_possible)

    while len(seen) < count:
        combo = tuple(sorted(rng.sample(range(n), d)))

        if combo in seen:
            continue

        seen.add(combo)
        yield combo



def chunked(iterable, size: int):
    batch = []

    for item in iterable:
        batch.append(item)

        if len(batch) >= size:
            yield batch
            batch = []
```

```
    if batch:
        yield batch



def expected_truth_table(func) -> Dict[str, Any]:
    table = {}

    for x in func.all_inputs():
        key = "".join(str(i) for i in x)
        y = func.eval(x)
        table[key] = "".join(str(i) for i in y)

    return table



def select_ranked_candidates(
    combos: List[Tuple[int, ...]],
    target: Optional[int],
    min_margin: Optional[float],
) -> Tuple[List[Tuple[int, ...]], List[Dict[str, Any]]]:
    scored = []

    for combo in combos:
        margin = fast_energy_margin(combo)
        scored.append((margin, combo))

    scored.sort(key=lambda t: (-t[0], t[1]))

    selected = []
    rows = []
```

```
    for margin, combo in scored:
        if min_margin is not None and margin < min_margin:
            continue

        if target is not None and target > 0 and len(selected) >= target:
            break

        selected.append(combo)

        rows.append(
            {
                "rank": len(selected),
                "energy_margin": margin,
                "canvas_indices": " ".join(str(i) for i in combo),
            }
        )

    return selected, rows


def write_csv_simple(path: str, rows: List[Dict[str, Any]]):
    if not rows:
        return

    os.makedirs(os.path.dirname(path), exist_ok=True)

    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)
```

```
def write_validation_sheet(
    root_dir: str,
    gate: str,
    candidate_ids: List[int],
    func,
    candidate_file_map: Dict[int, str],
):
    path = os.path.join(root_dir, "validation_sheet.csv")

    fieldnames = [
        "gate",
        "candidate_id",
        "input_bits",
        "expected_output",
        "observed_output",
        "io_integrity_ok",
        "siqad_file",
        "notes",
    ]

    rows = []

    for cid in candidate_ids:
        for x in func.all_inputs():
            input_bits = "".join(str(i) for i in x)
            expected = "".join(str(i) for i in func.eval(x))

            rows.append(
                {
                    "gate": gate,
```

```
                "candidate_id": cid,
                "input_bits": input_bits,
                "expected_output": expected,
                "observed_output": "",
                "io_integrity_ok": "",
                "siqad_file": candidate_file_map[cid],
                "notes": "",
            }
        )

    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)

    return path


def write_readme(root_dir: str, gate: str):
    path = os.path.join(root_dir, "README_validation.md")

    lines = [
        f"# SiQAD Validation Package for {gate}",
        "",
        "Open `candidate.sqd` in each candidate folder.",
        "",
        "Use `all_sidbs.csv` to identify IN/OUT BDL pairs.",
        "",
        "Fill `validation_sheet.csv` after SiQAD simulation.",
        "",
        "BDL encoding used by this prototype:",
```

```
        "",
        "- bit 0: pos_a = -1, pos_b = 0",
        "- bit 1: pos_a = 0, pos_b = -1",
        "",
        "After filling the validation sheet, run:",
        "",
        "```bash",
        "python analyze_siqad_results.py --sheet validation_exports/<GATE>/validation_sheet.csv",
        "```",
    ]

    with open(path, "w", encoding="utf-8") as f:
        f.write("\n".join(lines))

    return path


def parse_gate_argument(gate_arg: str) -> List[str]:
    gate_arg = gate_arg.strip()

    if gate_arg.upper() == "ALL":
        return get_all_3input_gate_names()

    return [
        normalize_gate_name(g)
        for g in gate_arg.split(",")
        if g.strip()
    ]


def run_export_for_gate(args, gate: str) -> Dict[str, Any]:
```

```
gate = normalize_gate_name(gate)

func = get_3input_gate(gate)
skeleton = paper_like_skeleton_3in_1out_25()
canvas = paper_like_canvas_143()

params = make_params(
    profile=args.strict_profile,
    io_margin_override=args.io_margin,
    pressure_margin_override=args.pressure_margin,
    wire_radius_override=args.wire_radius,
    input_disturbance_override=args.input_disturbance,
    connectivity_radius_override=args.connectivity_radius,
    charge_min_fraction_override=args.charge_min_fraction,
    charge_max_fraction_override=args.charge_max_fraction,
    energy_margin_override=args.energy_margin,
)

params = apply_runtime_overrides(
    params=params,
    d_min=args.d_min,
    positive_charge_margin=args.positive_charge_margin,
    charge_count_conflict_radius=args.charge_count_conflict_radius,
    f11_mode=args.f11_mode,
    f11_min_support_states=args.f11_min_support_states,
)

root_dir = os.path.join(args.output_dir, gate)
os.makedirs(root_dir, exist_ok=True)

combos = random_combinations(
```

```
    n=canvas.n_pos,
    d=4,
    count=args.samples,
    seed=args.seed,
)

chunks = list(chunked(combos, args.chunksize))

print("=" * 80)
print(f"Exporting SiQAD validation candidates for gate: {gate}")
print(f"|C| = {canvas.n_pos}, d = 4, |Ld| = {math.comb(canvas.n_pos, 4):,}")
print(f"Samples: {args.samples}")
print(f"Max candidates to export: {args.max_candidates}")
print(f"Workers: {args.workers}")
print(f"Chunksize: {args.chunksize}")
print(f"Seed: {args.seed}")
print(f"Profile: {args.strict_profile}")
print("-" * 80)
print(f"F1 d_min = {params.d_min}")
print(f"F3 wire forbidden radius = {params.wire_forbidden_radius}")
print(f"F4 positive charge margin = {params.positive_charge_margin}")
print(f"F5 charge fraction = [{params.charge_min_fraction}, {params.charge_max_fraction}]")
print(f"F5 charge-count conflict radius = {params.charge_count_conflict_radius}")
print(f"F6 input disturbance limit = {params.input_disturbance_limit}")
print(f"F7 connectivity radius = {params.connectivity_radius}")
print(f"F9 pressure margin = {params.pressure_margin}")
print(f"F10 energy margin = {params.energy_bound_margin}")
print(f"F11 mode = {params.f11_mode}")
print(f"F11 min support states = {params.f11_min_support_states}")
print(f"F11 relaxed enumeration = {params.relaxed_enumeration}")
print(f"F11 hop stability check = {params.check_configuration_stability}")
```

```
print(f"F12 I/O margin = {params.io_instability_margin}")
print("-" * 80)
print(f"Selection target: {args.selection_target}")
print(f"Selection min margin: {args.selection_min_margin}")
print("=" * 80)

total_processed = 0
after_counts = [0 for _ in range(12)]
passed_all = []

start = time.time()

if args.workers <= 1:
    _init_worker(skeleton, canvas, params, func)

    for chunk in chunks:
        result = worker_process_chunk(chunk)

        total_processed += result["processed"]

        for i in range(12):
            after_counts[i] += result["qc12_after"][i]

        passed_all.extend(result["qc12_passed"])

        print(
            f"[{gate}] processed {total_processed}/{args.samples}, "
            f"raw passed {len(passed_all)}"
        )

else:
```

```
    ctx = mp.get_context("spawn")

    with ctx.Pool(
        processes=args.workers,
        initializer=_init_worker,
        initargs=(skeleton, canvas, params, func),
    ) as pool:
        for idx, result in enumerate(
            pool.imap_unordered(worker_process_chunk, chunks),
            1,
        ):
            total_processed += result["processed"]

            for i in range(12):
                after_counts[i] += result["qc12_after"][i]

            passed_all.extend(result["qc12_passed"])

            if idx % 5 == 0 or idx == len(chunks):
                print(
                    f"[{gate}] processed {total_processed}/{args.samples}, "
                    f"raw passed {len(passed_all)}"
                )

# Reinitialize in main process for ranking/export.
_init_worker(skeleton, canvas, params, func)

ranking_rows: List[Dict[str, Any]] = []

selection_enabled = (
    args.selection_target > 0
```

```
    or args.selection_min_margin is not None
)

if selection_enabled:
    selected_combos, ranking_rows = select_ranked_candidates(
        combos=passed_all,
        target=args.selection_target if args.selection_target > 0 else None,
        min_margin=args.selection_min_margin,
    )
else:
    selected_combos = passed_all[: args.max_candidates]

if len(selected_combos) > args.max_candidates:
    selected_combos = selected_combos[: args.max_candidates]

candidate_ids: List[int] = []
candidate_file_map: Dict[int, str] = {}
summary_rows: List[Dict[str, Any]] = []

truth = expected_truth_table(func)

for combo in selected_combos:
    cid = len(candidate_ids) + 1

    layout = Layout(
        skeleton=skeleton,
        canvas_positions=[canvas.positions[i] for i in combo],
        canvas_indices=combo,
    )

    files = write_candidate_export(
```

```
        root_dir=root_dir,
        gate_name=gate,
        candidate_id=cid,
        layout=layout,
        expected_truth_table=truth,
        snap_to_lattice=not args.no_snap,
    )

    candidate_ids.append(cid)
    candidate_file_map[cid] = files["sqd"]

    margin = fast_energy_margin(combo)

    summary_rows.append(
        {
            "gate": gate,
            "candidate_id": cid,
            "energy_margin": margin,
            "canvas_indices": " ".join(str(i) for i in combo),
            "candidate_json": files["json"],
            "candidate_csv": files["csv"],
            "candidate_sqd": files["sqd"],
        }
    )

sheet_path = write_validation_sheet(
    root_dir=root_dir,
    gate=gate,
    candidate_ids=candidate_ids,
    func=func,
    candidate_file_map=candidate_file_map,
```

```
)

write_csv_simple(
    os.path.join(root_dir, "export_summary.csv"),
    summary_rows,
)

if ranking_rows:
    write_csv_simple(
        os.path.join(root_dir, "candidate_ranking.csv"),
        ranking_rows,
    )

readme_path = write_readme(root_dir, gate)

elapsed = time.time() - start

report = {
    "gate": gate,
    "samples_requested": args.samples,
    "samples_processed": total_processed,
    "raw_post_f12_candidates": len(passed_all),
    "exported_candidates": len(candidate_ids),
    "after_counts": {
        f"F{i + 1}": after_counts[i]
        for i in range(12)
    },
    "elapsed_seconds": elapsed,
    "validation_sheet": sheet_path,
    "readme": readme_path,
    "profile": args.strict_profile,
```

```
    "parameters": {
        "d_min": params.d_min,
        "wire_forbidden_radius": params.wire_forbidden_radius,
        "positive_charge_margin": params.positive_charge_margin,
        "charge_min_fraction": params.charge_min_fraction,
        "charge_max_fraction": params.charge_max_fraction,
        "charge_count_conflict_radius": params.charge_count_conflict_radius,
        "input_disturbance_limit": params.input_disturbance_limit,
        "connectivity_radius": params.connectivity_radius,
        "pressure_margin": params.pressure_margin,
        "energy_bound_margin": params.energy_bound_margin,
        "f11_mode": params.f11_mode,
        "f11_min_support_states": params.f11_min_support_states,
        "f11_relaxed_enumeration": params.relaxed_enumeration,
        "f11_hop_stability_check": params.check_configuration_stability,
        "io_instability_margin": params.io_instability_margin,
    },
}

with open(
    os.path.join(root_dir, "export_report.json"),
    "w",
    encoding="utf-8",
) as f:
    json.dump(report, f, indent=2, ensure_ascii=False)

print("\nFilter counts:")
for i, v in enumerate(after_counts, start=1):
    print(f" After F{i}: {v}")

print(f"\nExport completed for {gate}.")
```

```
    print(f"Output folder: {root_dir}")
    print(f"Validation sheet: {sheet_path}")
    print(f"Raw post-F12 candidates: {len(passed_all)}")
    print(f"Exported candidates: {len(candidate_ids)}")
    print(f"Elapsed time: {elapsed:.2f}s")

    return report


def write_global_summary(output_dir: str, reports: List[Dict[str, Any]]):
    os.makedirs(output_dir, exist_ok=True)

    path = os.path.join(output_dir, "all_gates_export_summary.csv")

    fieldnames = [
        "gate",
        "samples_requested",
        "samples_processed",
        "raw_post_f12_candidates",
        "exported_candidates",
        "elapsed_seconds",
        "validation_sheet",
        "profile",
        "d_min",
        "io_margin",
        "connectivity_radius",
        "pressure_margin",
        "energy_margin",
        "f11_mode",
        "f11_min_support_states",
    ]
```

```
rows = []

for r in reports:
    p = r.get("parameters", {})

    rows.append(
        {
            "gate": r.get("gate"),
            "samples_requested": r.get("samples_requested"),
            "samples_processed": r.get("samples_processed"),
            "raw_post_f12_candidates": r.get("raw_post_f12_candidates"),
            "exported_candidates": r.get("exported_candidates"),
            "elapsed_seconds": r.get("elapsed_seconds"),
            "validation_sheet": r.get("validation_sheet"),
            "profile": r.get("profile"),
            "d_min": p.get("d_min"),
            "io_margin": p.get("io_instability_margin"),
            "connectivity_radius": p.get("connectivity_radius"),
            "pressure_margin": p.get("pressure_margin"),
            "energy_margin": p.get("energy_bound_margin"),
            "f11_mode": p.get("f11_mode"),
            "f11_min_support_states": p.get("f11_min_support_states"),
        }
    )

write_csv_simple(path, rows)

print(f"\nGlobal export summary: {path}")
```

```
def main():
    parser = argparse.ArgumentParser(
        description="Export QuickCell candidates for SiQAD validation."
    )

    parser.add_argument(
        "--gate",
        required=True,
        help="Gate name, comma-separated list, or ALL.",
    )

    parser.add_argument("--samples", type=int, default=100000)
    parser.add_argument("--max-candidates", type=int, default=20)
    parser.add_argument("--workers", type=int, default=4)
    parser.add_argument("--chunksize", type=int, default=2000)
    parser.add_argument("--seed", type=int, default=12345)
    parser.add_argument("--output-dir", default="validation_exports")

    parser.add_argument(
        "--strict-profile",
        default="balanced",
        choices=["safe", "balanced", "strict"],
    )

    # Existing / original margins
    parser.add_argument(
        "--io-margin",
        type=float,
        default=None,
        help="Override F12 I/O instability margin. Smaller is stricter.",
    )
```

```
parser.add_argument(
    "--pressure-margin",
    type=float,
    default=None,
    help="Override F9 pressure margin. Larger is stricter.",
)

# Added tunable filters
parser.add_argument(
    "--d-min",
    type=float,
    default=None,
    help="Override F1 minimum canvas-canvas distance. Larger is stricter.",
)

parser.add_argument(
    "--wire-radius",
    type=float,
    default=None,
    help="Override F3 wire forbidden radius. Larger is stricter.",
)

parser.add_argument(
    "--positive-charge-margin",
    type=float,
    default=0.0,
    help="F4 positive-charge safety margin. Larger is stricter.",
)

parser.add_argument(
```

```
    "--charge-min-fraction",
    type=float,
    default=None,
    help="Override F5 minimum negative-charge fraction.",
)

parser.add_argument(
    "--charge-max-fraction",
    type=float,
    default=None,
    help="Override F5 maximum negative-charge fraction.",
)

parser.add_argument(
    "--charge-count-conflict-radius",
    type=float,
    default=None,
    help="F5 canvas negative-charge conflict radius. Larger is stricter.",
)

parser.add_argument(
    "--input-disturbance",
    type=float,
    default=None,
    help="Override F6 input disturbance limit. Smaller is stricter.",
)

parser.add_argument(
    "--connectivity-radius",
    type=float,
    default=None,
```

```
    help="Override F7 connectivity radius. Smaller is stricter.",
)

parser.add_argument(
    "--energy-margin",
    type=float,
    default=None,
    help="Override F10 relaxed energy margin. Smaller is stricter.",
)

parser.add_argument(
    "--f11-mode",
    default="counted",
    choices=["counted", "strict-pop", "strict-hop"],
    help=(
        "F11 execution mode. "
        "counted = do not run F11; "
        "strict-pop = population stability only; "
        "strict-hop = population + hop stability."
    ),
)

parser.add_argument(
    "--f11-min-support-states",
    type=int,
    default=1,
    help=(
        "F11 minimum number of metastable supporting canvas states "
        "per input. Larger is stricter."
    ),
)
```

```
# Candidate selection / export controls
parser.add_argument(
    "--selection-target",
    type=int,
    default=0,
    help="Rank post-F12 candidates and export only top K. 0 disables.",
)

parser.add_argument(
    "--selection-min-margin",
    type=float,
    default=None,
    help="Optional minimum energy margin for exported candidates.",
)

parser.add_argument(
    "--no-snap",
    action="store_true",
    help="Do not snap exported SiDBs to nearest SiQAD lattice coordinates.",
)

args = parser.parse_args()

gates = parse_gate_argument(args.gate)

reports = []

for gate in gates:
    report = run_export_for_gate(args, gate)
    reports.append(report)
```

```
    write_global_summary(args.output_dir, reports)



if __name__ == "__main__":
    main()
```

---

```
# analyze_siqad_results.py

from __future__ import annotations

import argparse
import csv
import os
from collections import defaultdict
from glob import glob



TRUE_VALUES = {
    "1",
    "true",
    "yes",
    "y",
    "ok",
    "pass",
    "passed",
}

FALSE_VALUES = {
    "0",
    "false",
```

```
    "no",
    "n",
    "fail",
    "failed",
}



def parse_bool(value: str):
    value = str(value).strip().lower()

    if value in TRUE_VALUES:
        return True

    if value in FALSE_VALUES:
        return False

    return None



def is_blank_row(row: dict) -> bool:
    if row is None:
        return True

    for v in row.values():
        if str(v).strip() != "":
            return False

    return True



def safe_int_key(value: str):
```

```
    value = str(value).strip()

    try:
        return (0, int(value))
    except Exception:
        return (1, value)



def analyze_sheet(path: str, quiet: bool = False):
    rows = []
    warnings = []

    with open(path, "r", newline="", encoding="utf-8-sig") as f:
        reader = csv.DictReader(f)

        required_columns = {
            "gate",
            "candidate_id",
            "input_bits",
            "expected_output",
            "observed_output",
            "io_integrity_ok",
            "siqad_file",
            "notes",
        }

        if reader.fieldnames is None:
            raise RuntimeError(f"CSV file has no header row: {path}")

        missing_cols = required_columns - set(reader.fieldnames)
```

```
    if missing_cols:
        raise RuntimeError(
            f"CSV file {path} is missing required columns: "
            + ", ".join(sorted(missing_cols))
        )

    for line_no, row in enumerate(reader, start=2):
        if is_blank_row(row):
            continue

        cid = str(row.get("candidate_id", "")).strip()

        if cid == "":
            warnings.append(
                f"Line {line_no}: skipped row with empty candidate_id."
            )
            continue

        input_bits = str(row.get("input_bits", "")).strip()

        if input_bits == "":
            warnings.append(
                f"Line {line_no}: skipped row with empty input_bits."
            )
            continue

        rows.append(row)

if not rows:
    if not quiet:
        print(f"No valid rows found in validation sheet: {path}")
```

```
    return {
        "sheet": path,
        "gate": "",
        "valid_rows": 0,
        "filled_rows": 0,
        "candidates": 0,
        "passed_candidates": 0,
        "failed_or_incomplete_candidates": 0,
        "summary_path": "",
    }

by_candidate = defaultdict(list)

for row in rows:
    cid = str(row["candidate_id"]).strip()
    by_candidate[cid].append(row)

summary = []
gate_name = str(rows[0].get("gate", "")).strip()

for cid, items in sorted(by_candidate.items(), key=lambda x: safe_int_key(x[0])):
    total = len(items)
    filled = 0
    correct = 0
    integrity_ok_count = 0
    missing = False

    failed_inputs = []
    incomplete_inputs = []

    for row in items:
```

```
input_bits = str(row.get("input_bits", "")).strip()
expected = str(row.get("expected_output", "")).strip()
observed = str(row.get("observed_output", "")).strip()

integrity_raw = str(row.get("io_integrity_ok", "")).strip()
integrity = parse_bool(integrity_raw)

if observed == "":
    missing = True
    incomplete_inputs.append(input_bits)
    continue

if integrity is None:
    missing = True
    incomplete_inputs.append(input_bits)
    continue

filled += 1

output_correct = observed == expected

if output_correct:
    correct += 1
else:
    failed_inputs.append(input_bits)

if integrity:
    integrity_ok_count += 1
else:
    if input_bits not in failed_inputs:
        failed_inputs.append(input_bits)
```

```
    pass_candidate = (
        not missing
        and filled == total
        and correct == total
        and integrity_ok_count == total
    )

    summary.append({
        "gate": gate_name,
        "candidate_id": cid,
        "total_inputs": total,
        "filled_inputs": filled,
        "correct_outputs": correct,
        "io_integrity_ok": integrity_ok_count,
        "candidate_pass": pass_candidate,
        "failed_inputs": " ".join(failed_inputs),
        "incomplete_inputs": " ".join(incomplete_inputs),
    })

out_dir = os.path.dirname(path)

summary_path = os.path.join(
    out_dir,
    "siqad_validation_summary.csv",
)

with open(summary_path, "w", newline="", encoding="utf-8") as f:
    fieldnames = [
        "gate",
        "candidate_id",
```

```
        "total_inputs",
        "filled_inputs",
        "correct_outputs",
        "io_integrity_ok",
        "candidate_pass",
        "failed_inputs",
        "incomplete_inputs",
    ]

    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(summary)

total_candidates = len(summary)
passed = sum(1 for row in summary if row["candidate_pass"])
failed_or_incomplete = total_candidates - passed

total_rows = len(rows)
filled_rows = sum(
    1
    for row in rows
    if str(row.get("observed_output", "")).strip() != ""
    and parse_bool(row.get("io_integrity_ok", "")) is not None
)

result = {
    "sheet": path,
    "gate": gate_name,
    "valid_rows": total_rows,
    "filled_rows": filled_rows,
    "candidates": total_candidates,
```

```
        "passed_candidates": passed,
        "failed_or_incomplete_candidates": failed_or_incomplete,
        "summary_path": summary_path,
    }

    if not quiet:
        print("=" * 80)
        print("SiQAD validation analysis")
        print(f"Sheet: {path}")
        print(f"Gate: {gate_name}")
        print(f"Valid input rows: {total_rows}")
        print(f"Filled rows: {filled_rows}")
        print(f"Candidates: {total_candidates}")
        print(f"Passed candidates: {passed}")
        print(f"Failed or incomplete candidates: {failed_or_incomplete}")
        print(f"Summary CSV: {summary_path}")
        print("=" * 80)

    return result


def analyze_all(root: str):
    pattern = os.path.join(root, "*", "validation_sheet.csv")
    sheets = sorted(glob(pattern))

    if not sheets:
        print(f"No validation_sheet.csv files found under: {root}")
        return

    results = []
```

```
for sheet in sheets:
    res = analyze_sheet(sheet, quiet=True)
    results.append(res)

out_path = os.path.join(root, "all_gates_siqad_validation_summary.csv")

fieldnames = [
    "gate",
    "sheet",
    "valid_rows",
    "filled_rows",
    "candidates",
    "passed_candidates",
    "failed_or_incomplete_candidates",
    "summary_path",
]

with open(out_path, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()

    for r in results:
        writer.writerow({k: r.get(k, "") for k in fieldnames})

print("=" * 80)
print("All-gate SiQAD validation summary")
print(f"Root: {root}")
print(f"Sheets analyzed: {len(sheets)}")
print(f"Combined summary: {out_path}")
print("=" * 80)
```

```
    for r in results:
        print(
            f"{r['gate']}: "
            f"passed {r['passed_candidates']}/{r['candidates']} candidates, "
            f"filled rows {r['filled_rows']}/{r['valid_rows']}"
        )


def main():
    parser = argparse.ArgumentParser(
        description="Analyze filled SiQAD validation sheet(s)."
    )

    parser.add_argument("--sheet", default="")
    parser.add_argument("--root", default="validation_exports")
    parser.add_argument("--all", action="store_true")

    args = parser.parse_args()

    if args.all:
        analyze_all(args.root)
    else:
        if not args.sheet:
            raise RuntimeError("Please provide --sheet or use --all.")

        analyze_sheet(args.sheet)


if __name__ == "__main__":
    main()
```

---

```
# src/__init__.py

from .data_structures import SiDB, Skeleton, Canvas, Layout, BooleanFunction
from .physics import PhysicalParameters
from .benchmarks import skeleton_2in_1out, skeleton_3in_1out, grid_canvas

__all__ = [
    "SiDB",
    "Skeleton",
    "Canvas",
    "Layout",
    "BooleanFunction",
    "PhysicalParameters",
    "skeleton_2in_1out",
    "skeleton_3in_1out",
    "grid_canvas",
]
```

---

```
# src/benchmarks.py

from __future__ import annotations

from typing import List, Tuple

import numpy as np

from .data_structures import Skeleton


def skeleton_2in_1out(bdl_spacing: float = 0.76) -> Skeleton:
    s = float(bdl_spacing)
```

```
    return Skeleton(
        input_pins_coords=[
            [(-5.26, 2.50), (-5.26 + s, 2.50)],
            [(-5.26, -2.50), (-5.26 + s, -2.50)],
        ],
        output_pins_coords=[
            [(4.50, 0.00), (4.50 + s, 0.00)],
        ],
        wire_coords=[],
    )


def skeleton_3in_1out(bdl_spacing: float = 0.76) -> Skeleton:
    s = float(bdl_spacing)

    return Skeleton(
        input_pins_coords=[
            [(-6.26, 3.50), (-6.26 + s, 3.50)],
            [(-6.26, 0.00), (-6.26 + s, 0.00)],
            [(-6.26, -3.50), (-6.26 + s, -3.50)],
        ],
        output_pins_coords=[
            [(5.50, 0.00), (5.50 + s, 0.00)],
        ],
        wire_coords=[],
    )


def grid_canvas(
    x_range: Tuple[float, float],
```

```
    y_range: Tuple[float, float],
    step: float,
) -> List[Tuple[float, float]]:
    positions: List[Tuple[float, float]] = []

    x0, x1 = x_range
    y0, y1 = y_range

    for x in np.arange(x0, x1 + 0.5 * step, step):
        for y in np.arange(y0, y1 + 0.5 * step, step):
            positions.append((float(x), float(y)))

    return positions
--------------------------------------------------------------------------------
# src/data_structures.py

from __future__ import annotations

from itertools import product
from typing import Callable, Iterable, List, Optional, Sequence, Tuple, Dict, Any

import numpy as np



def _and2_func(x: Tuple[int, int]) -> int:
    return int(x[0] & x[1])



def _or2_func(x: Tuple[int, int]) -> int:
    return int(x[0] | x[1])
```

```
def _xor2_func(x: Tuple[int, int]) -> int:
    return int(x[0] ^ x[1])


def _const0_2_func(x: Tuple[int, int]) -> int:
    return 0


def _const1_2_func(x: Tuple[int, int]) -> int:
    return 1


def _and3_func(x: Tuple[int, int, int]) -> int:
    return int(x[0] & x[1] & x[2])


def _xor3_func(x: Tuple[int, int, int]) -> int:
    return int(x[0] ^ x[1] ^ x[2])


def _maj3_func(x: Tuple[int, int, int]) -> int:
    return int((x[0] + x[1] + x[2]) >= 2)


def as_3d_array(p: Sequence[float]) -> np.ndarray:
    if len(p) == 2:
        return np.array([float(p[0]), float(p[1]), 0.0], dtype=float)

    if len(p) >= 3:
        return np.array([float(p[0]), float(p[1]), float(p[2])], dtype=float)
```

```
    raise ValueError("Coordinate must have length 2 or 3.")



def coord_key(
    p: Sequence[float] | np.ndarray,
    ndigits: int = 9,
) -> Tuple[float, float, float]:
    a = np.asarray(p, dtype=float)

    return (
        round(float(a[0]), ndigits),
        round(float(a[1]), ndigits),
        round(float(a[2]), ndigits),
    )



class SiDB:
    def __init__(self, x: float, y: float, z: float = 0.0):
        self.coords = np.array([float(x), float(y), float(z)], dtype=float)



class Skeleton:
    class Pin:
        def __init__(self, index: int, sidbs: List[SiDB]):
            if len(sidbs) != 2:
                raise ValueError("Each BDL pin must contain exactly two SiDB coordinates.")

            self.pin_index = index
            self.sidbs = sidbs
            self.pos_a = sidbs[0].coords
```

```
        self.pos_b = sidbs[1].coords

    def charges_for_bit(self, bit: int) -> List[Tuple[np.ndarray, int]]:
        bit = int(bit)

        if bit == 0:
            return [(self.pos_a, -1), (self.pos_b, 0)]

        if bit == 1:
            return [(self.pos_a, 0), (self.pos_b, -1)]

        raise ValueError("BDL bit must be 0 or 1.")

    def positions(self) -> List[np.ndarray]:
        return [self.pos_a, self.pos_b]

def __init__(
    self,
    input_pins_coords: List[List[Tuple[float, ...]]],
    output_pins_coords: List[List[Tuple[float, ...]]],
    wire_coords: Optional[List[Tuple[float, ...]]] = None,
):
    wire_coords = wire_coords or []

    self.n_in = len(input_pins_coords)
    self.n_out = len(output_pins_coords)

    self.input_pins = [
        self._create_pin(i, coords)
        for i, coords in enumerate(input_pins_coords)
    ]
```

```
    self.output_pins = [
        self._create_pin(i, coords)
        for i, coords in enumerate(output_pins_coords)
    ]

    self.wire_sidbs = []

    for p in wire_coords:
        a = as_3d_array(p)
        self.wire_sidbs.append(SiDB(a[0], a[1], a[2]))

    self.all_sidbs = (
        [sdb for pin in self.input_pins for sdb in pin.sidbs]
        + [sdb for pin in self.output_pins for sdb in pin.sidbs]
        + self.wire_sidbs
    )

def _create_pin(
    self,
    index: int,
    coords_list: List[Tuple[float, ...]],
) -> Pin:
    sidbs = []

    for p in coords_list:
        a = as_3d_array(p)
        sidbs.append(SiDB(a[0], a[1], a[2]))

    return self.Pin(index, sidbs)
```

```
def input_charge_config(
    self,
    x: Tuple[int, ...],
) -> List[Tuple[np.ndarray, int]]:
    if len(x) != self.n_in:
        raise ValueError(f"Expected {self.n_in} input bits, got {len(x)}.")

    config: List[Tuple[np.ndarray, int]] = []

    for i, pin in enumerate(self.input_pins):
        config.extend(pin.charges_for_bit(int(x[i])))

    return config

def output_charge_config(
    self,
    y: Tuple[int, ...],
) -> List[Tuple[np.ndarray, int]]:
    if len(y) != self.n_out:
        raise ValueError(f"Expected {self.n_out} output bits, got {len(y)}.")

    config: List[Tuple[np.ndarray, int]] = []

    for i, pin in enumerate(self.output_pins):
        config.extend(pin.charges_for_bit(int(y[i])))

    return config

def io_charge_config(
    self,
    x: Tuple[int, ...],
```

```
        y: Tuple[int, ...],
    ) -> List[Tuple[np.ndarray, int]]:
        return self.input_charge_config(x) + self.output_charge_config(y)

    def input_positions(self) -> List[np.ndarray]:
        return [p for pin in self.input_pins for p in pin.positions()]

    def output_positions(self) -> List[np.ndarray]:
        return [p for pin in self.output_pins for p in pin.positions()]

    def all_positions(self) -> List[np.ndarray]:
        return [s.coords for s in self.all_sidbs]



class Canvas:
    def __init__(self, positions: List[Tuple[float, ...]]):
        self.positions = [as_3d_array(p) for p in positions]
        self.n_pos = len(self.positions)

        self.position_to_index = {}

        for i, p in enumerate(self.positions):
            k = coord_key(p)

            if k in self.position_to_index:
                raise ValueError(f"Duplicate canvas position detected: {k}")

            self.position_to_index[k] = i

    def index_of(self, p: Sequence[float] | np.ndarray) -> Optional[int]:
        return self.position_to_index.get(coord_key(p), None)
```

```
class Layout:
    def __init__(
        self,
        skeleton: Skeleton,
        canvas_positions: Optional[List[np.ndarray]] = None,
        canvas_indices: Optional[Sequence[Optional[int]]] = None,
    ):
        self.skeleton = skeleton
        self.canvas_positions = list(canvas_positions or [])

        if canvas_indices is None:
            self.canvas_indices: Tuple[Optional[int], ...] = tuple(
                [None] * len(self.canvas_positions)
            )
        else:
            self.canvas_indices = tuple(canvas_indices)

        self.all_positions = (
            [s.coords for s in skeleton.all_sidbs]
            + self.canvas_positions
        )

        self.n_canvas = len(self.canvas_positions)
        self.cache: Dict[Any, Any] = {}

    def add_canvas_sdb(
        self,
        position: np.ndarray,
        index: Optional[int] = None,
```

```
    ) -> "Layout":
        return Layout(
            skeleton=self.skeleton,
            canvas_positions=self.canvas_positions + [np.asarray(position, dtype=float)],
            canvas_indices=self.canvas_indices + (index,),
        )

    def canvas_key(self) -> Tuple:
        if all(i is not None for i in self.canvas_indices):
            return tuple(sorted(int(i) for i in self.canvas_indices if i is not None))

        return tuple(sorted(coord_key(p) for p in self.canvas_positions))


class BooleanFunction:
    def __init__(
        self,
        name: str,
        num_inputs: int,
        func: Callable[[Tuple[int, ...]], int | Tuple[int, ...]],
        num_outputs: int = 1,
    ):
        self.name = name
        self.num_inputs = int(num_inputs)
        self.num_outputs = int(num_outputs)
        self._func = func

    def eval(self, inputs: Tuple[int, ...]) -> Tuple[int, ...]:
        if len(inputs) != self.num_inputs:
            raise ValueError(
                f"{self.name}: expected {self.num_inputs} inputs, got {len(inputs)}."
```

```
        )

    res = self._func(tuple(int(v) for v in inputs))

    if isinstance(res, (int, bool, np.integer)):
        return (int(res),)

    return tuple(int(v) for v in res)

def all_inputs(self) -> Iterable[Tuple[int, ...]]:
    return product([0, 1], repeat=self.num_inputs)

def all_outputs(self) -> Iterable[Tuple[int, ...]]:
    return product([0, 1], repeat=self.num_outputs)

@classmethod
def AND2(cls) -> "BooleanFunction":
    return cls("AND2", 2, _and2_func, 1)

@classmethod
def OR2(cls) -> "BooleanFunction":
    return cls("OR2", 2, _or2_func, 1)

@classmethod
def XOR2(cls) -> "BooleanFunction":
    return cls("XOR2", 2, _xor2_func, 1)

@classmethod
def CONST0_2(cls) -> "BooleanFunction":
    return cls("CONST0_2", 2, _const0_2_func, 1)
```

```
    @classmethod
    def CONST1_2(cls) -> "BooleanFunction":
        return cls("CONST1_2", 2, _const1_2_func, 1)

    @classmethod
    def AND3(cls) -> "BooleanFunction":
        return cls("AND3", 3, _and3_func, 1)

    @classmethod
    def XOR3(cls) -> "BooleanFunction":
        return cls("XOR3", 3, _xor3_func, 1)

    @classmethod
    def MAJ3(cls) -> "BooleanFunction":
        return cls("MAJ3", 3, _maj3_func, 1)
```

---

```
# src/filters.py

from __future__ import annotations

import math
from collections import deque
from itertools import product
from typing import Dict, List, Optional, Tuple

import numpy as np

from .data_structures import (
    BooleanFunction,
    Canvas,
    Layout,
```

```
    Skeleton,
    coord_key,
)
from .physics import (
    PhysicalParameters,
    euclidean,
    get_potential,
    get_total_energy,
)
from .stability import (
    interval_intersects_charge_state,
    is_metastable,
    is_population_stable_charge,
)



def _config_positions_charges(
    config: List[Tuple[np.ndarray, int]],
) -> Tuple[List[np.ndarray], List[int]]:
    return [p for p, _ in config], [int(c) for _, c in config]



def _fixed_config_for_io(
    skeleton: Skeleton,
    x: Tuple[int, ...],
    y: Tuple[int, ...],
) -> List[Tuple[np.ndarray, int]]:
    config = skeleton.io_charge_config(x, y)

    for w in skeleton.wire_sidbs:
        config.append((w.coords, 0))
```

```
    return config



def _potential_from_sources(
    target: np.ndarray,
    source_positions: List[np.ndarray],
    source_charges: List[int],
    params: PhysicalParameters,
    exclude_keys: Optional[set] = None,
) -> float:
    exclude_keys = exclude_keys or set()
    t_key = coord_key(target)

    v = 0.0

    for p, q in zip(source_positions, source_charges):
        p_key = coord_key(p)

        if p_key == t_key:
            continue

        if p_key in exclude_keys:
            continue

        v += int(q) * get_potential(target, p, params)

    return float(v)



def _canvas_free_indices(
```

```
    n_fixed: int,
    n_total: int,
    params: PhysicalParameters,
):
    if params.check_fixed_io_population:
        return None

    return range(n_fixed, n_total)



def _permute_tuple_old_to_new(
    values: Tuple[int, ...],
    old_to_new: Tuple[int, ...],
) -> Tuple[int, ...]:
    out = [0] * len(values)

    for old, new in enumerate(old_to_new):
        out[new] = values[old]

    return tuple(out)



def is_valid_partial_f1(
    layout: Layout,
    params: PhysicalParameters,
    **kwargs,
) -> bool:
    pts = layout.canvas_positions

    for i in range(len(pts)):
        for j in range(i + 1, len(pts)):
```

```
            if euclidean(pts[i], pts[j]) < params.d_min:
                return False

    return True



def F1_min_distance(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction | None = None,
    skeleton: Skeleton | None = None,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    return not is_valid_partial_f1(layout, params)



_SYMMETRY_CACHE: Dict[Tuple[int, int, str, Tuple], List[List[int]]] = {}



def _transform_point(
    p: np.ndarray,
    name: str,
    center: Tuple[float, float],
) -> np.ndarray:
    x, y, z = float(p[0]), float(p[1]), float(p[2])
    cx, cy = center

    if name == "identity":
        return np.array([x, y, z])

```

```
    if name == "reflect_x":
        return np.array([2 * cx - x, y, z])

    if name == "reflect_y":
        return np.array([x, 2 * cy - y, z])

    if name == "rotate_180":
        return np.array([2 * cx - x, 2 * cy - y, z])

    if name == "rotate_90":
        return np.array([cx - (y - cy), cy + (x - cx), z])

    if name == "rotate_270":
        return np.array([cx + (y - cy), cy - (x - cx), z])

    raise ValueError(f"Unknown transform {name}")



def _pin_mapping_for_transform(
    pins: List[Skeleton.Pin],
    name: str,
    center: Tuple[float, float],
) -> Optional[Tuple[int, ...]]:
    target = {}

    for pin in pins:
        target[(coord_key(pin.pos_a), coord_key(pin.pos_b))] = pin.pin_index

    mapping: List[int] = []

    for pin in pins:
```

```
        a_t = coord_key(_transform_point(pin.pos_a, name, center))
        b_t = coord_key(_transform_point(pin.pos_b, name, center))

        idx = target.get((a_t, b_t), None)

        if idx is None:
            return None

        mapping.append(idx)

    if len(set(mapping)) != len(mapping):
        return None

    return tuple(mapping)



def _wire_maps_to_itself(
    skeleton: Skeleton,
    name: str,
    center: Tuple[float, float],
) -> bool:
    if not skeleton.wire_sidbs:
        return True

    wire_set = {coord_key(w.coords) for w in skeleton.wire_sidbs}

    mapped = {
        coord_key(_transform_point(w.coords, name, center))
        for w in skeleton.wire_sidbs
    }
```

```
    return mapped == wire_set



def _function_invariant(
    func: BooleanFunction,
    input_perm_old_to_new: Tuple[int, ...],
    output_perm_old_to_new: Tuple[int, ...],
) -> bool:
    for x in func.all_inputs():
        x_t = _permute_tuple_old_to_new(x, input_perm_old_to_new)

        y = func.eval(x)
        y_t_expected = _permute_tuple_old_to_new(y, output_perm_old_to_new)

        if func.eval(x_t) != y_t_expected:
            return False

    return True



def _get_allowed_symmetry_perms(
    skeleton: Skeleton,
    canvas: Canvas,
    func: BooleanFunction,
) -> List[List[int]]:
    truth_key = tuple(func.eval(x) for x in func.all_inputs())
    cache_key = (id(skeleton), id(canvas), func.name, truth_key)

    if cache_key in _SYMMETRY_CACHE:
        return _SYMMETRY_CACHE[cache_key]
```

```
xs = [p[0] for p in canvas.positions]
ys = [p[1] for p in canvas.positions]

center = (
    (min(xs) + max(xs)) / 2.0,
    (min(ys) + max(ys)) / 2.0,
)

candidate_names = [
    "identity",
    "reflect_x",
    "reflect_y",
    "rotate_180",
    "rotate_90",
    "rotate_270",
]

canvas_map = canvas.position_to_index
allowed: List[List[int]] = []

for name in candidate_names:
    perm: List[int] = []
    ok = True

    for p in canvas.positions:
        p_t = _transform_point(p, name, center)
        idx = canvas_map.get(coord_key(p_t), None)

        if idx is None:
            ok = False
            break
```

```
            perm.append(idx)

        if not ok:
            continue

        in_perm = _pin_mapping_for_transform(skeleton.input_pins, name, center)
        out_perm = _pin_mapping_for_transform(skeleton.output_pins, name, center)

        if in_perm is None or out_perm is None:
            continue

        if not _wire_maps_to_itself(skeleton, name, center):
            continue

        if not _function_invariant(func, in_perm, out_perm):
            continue

        allowed.append(perm)

    if not allowed:
        allowed = [list(range(canvas.n_pos))]

    _SYMMETRY_CACHE[cache_key] = allowed

    return allowed


def _layout_canvas_indices(
    layout: Layout,
    canvas: Optional[Canvas],
```

```
) -> Optional[Tuple[int, ...]]:
    if all(i is not None for i in layout.canvas_indices):
        return tuple(sorted(int(i) for i in layout.canvas_indices if i is not None))

    if canvas is None:
        return None

    indices = []

    for p in layout.canvas_positions:
        idx = canvas.index_of(p)

        if idx is None:
            return None

        indices.append(idx)

    return tuple(sorted(indices))



def F2_symmetry(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction,
    skeleton: Skeleton,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    if canvas is None:
        return False
```

```
    current = _layout_canvas_indices(layout, canvas)

    if current is None:
        return False

    perms = _get_allowed_symmetry_perms(skeleton, canvas, func)

    orbit = []

    for perm in perms:
        mapped = tuple(sorted(perm[i] for i in current))
        orbit.append(mapped)

    canonical = min(orbit)

    return current != canonical



def _point_segment_distance(
    p: np.ndarray,
    a: np.ndarray,
    b: np.ndarray,
) -> float:
    ab = b - a
    denom = float(np.dot(ab, ab))

    if denom < 1e-18:
        return euclidean(p, a)

    t = float(np.dot(p - a, ab) / denom)
    t = max(0.0, min(1.0, t))
```

```
    nearest = a + t * ab

    return euclidean(p, nearest)



def F3_wire_interference(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction | None = None,
    skeleton: Skeleton | None = None,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    if skeleton is None or not skeleton.wire_sidbs:
        return False

    r_forbidden = params.wire_forbidden_radius

    if r_forbidden is None:
        r_forbidden = max(params.d_min, 0.5 * params.lambda_tf)

    wire_pts = [w.coords for w in skeleton.wire_sidbs]

    for c in layout.canvas_positions:
        for w in wire_pts:
            if euclidean(c, w) < r_forbidden:
                return True

        for a, b in zip(wire_pts[:-1], wire_pts[1:]):
            if _point_segment_distance(c, a, b) < r_forbidden:
```

```
            return True

    return False



def F4_positive_charge(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction | None = None,
    skeleton: Skeleton | None = None,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    """
    F4: Positive charge pruning.

    Return True  => discard layout
    Return False => keep layout

    The optional parameter params.positive_charge_margin makes the
    filter stricter:

       activation = mu_plus + qe * v_local

    Original condition:
       activation > 0

    Margin condition:
       activation + margin > 0
       equivalently activation > -margin
    """
```

```
    margin = float(getattr(params, "positive_charge_margin", 0.0))

    positions = layout.all_positions

    for i in range(len(positions)):
        v_local = 0.0

        for j in range(len(positions)):
            if i == j:
                continue

            # Worst-case assumption: all other SiDBs are negatively charged.
            v_local += -1 * get_potential(positions[i], positions[j], params)

        activation = params.mu_plus + params.qe * v_local

        if activation + margin > 0:
            return True

    return False



def _max_independent_canvas_capacity(
    canvas_positions: List[np.ndarray],
    conflict_radius: float,
) -> int:
    """
    Maximum number of canvas SiDBs that can be simultaneously negative
    if pairs closer than conflict_radius are treated as charge-count conflicts.

    This is exact for small d by brute-force subset enumeration.
```

```
In your experiments d = 4, so this is very cheap.
"""
d = len(canvas_positions)

if d == 0:
    return 0

conflict = [[False for _ in range(d)] for _ in range(d)]

for i in range(d):
    for j in range(i + 1, d):
        if euclidean(canvas_positions[i], canvas_positions[j]) < conflict_radius:
            conflict[i][j] = True
            conflict[j][i] = True

best = 0

for mask in range(1 << d):
    ok = True
    count = 0

    for i in range(d):
        if mask & (1 << i):
            count += 1

    if count <= best:
        continue

    for i in range(d):
        if not (mask & (1 << i)):
            continue
```

```
            for j in range(i + 1, d):
                if not (mask & (1 << j)):
                    continue

                if conflict[i][j]:
                    ok = False
                    break

            if not ok:
                break

        if ok:
            best = count

    return best



def F5_charge_count_bound(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction,
    skeleton: Skeleton,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    """
    F5: Charge-count bound pruning.

    Return True  => discard layout
    Return False => keep layout
```

```
Original behavior:
    req_max = required_skeleton_negative_count + number_of_canvas_sites

Improved layout-sensitive behavior:
    If params.charge_count_conflict_radius is set, nearby canvas SiDBs
    are treated as mutually conflicting for simultaneous negative charge.
    Then req_max uses the maximum independent-set capacity of the selected
    canvas sites.

This makes F5 capable of pruning some layouts without becoming an
all-or-nothing filter.
"""
n_total = len(layout.all_positions)

adm_min = math.ceil(params.charge_min_fraction * n_total)
adm_max = math.floor(params.charge_max_fraction * n_total)

adm_min = max(0, adm_min)
adm_max = min(n_total, adm_max)

conflict_radius = getattr(params, "charge_count_conflict_radius", None)

if conflict_radius is None or float(conflict_radius) <= 0.0:
    max_canvas_negative_capacity = layout.n_canvas
else:
    max_canvas_negative_capacity = _max_independent_canvas_capacity(
        layout.canvas_positions,
        float(conflict_radius),
    )
```

```
    for x in func.all_inputs():
        y = func.eval(x)

        io_config = _fixed_config_for_io(skeleton, x, y)

        n_req_skeleton = sum(
            1 for _, q in io_config
            if int(q) == -1
        )

        req_min = n_req_skeleton
        req_max = n_req_skeleton + max_canvas_negative_capacity

        if req_max < adm_min or req_min > adm_max:
            return True

    return False


def F6_input_pin_disturbance(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction,
    skeleton: Skeleton,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    if params.input_disturbance_limit is None:
        limit = abs(
            (-params.mu_minus / params.qe)
            - (-params.mu_plus / params.qe)
```

```
    )
else:
    limit = float(params.input_disturbance_limit)

for x in func.all_inputs():
    y = func.eval(x)

    full_skel_config = _fixed_config_for_io(skeleton, x, y)
    skel_pos, skel_chg = _config_positions_charges(full_skel_config)

    input_config = skeleton.input_charge_config(x)

    for site_pos, required_q in input_config:
        canvas_worst = sum(
            -get_potential(site_pos, c, params)
            for c in layout.canvas_positions
        )

        if params.enforce_input_population:
            base_v = _potential_from_sources(
                site_pos,
                skel_pos,
                skel_chg,
                params,
            )

            disturbed_v = base_v + canvas_worst

            if not is_population_stable_charge(
                required_q,
                disturbed_v,
```

```
                params,
            ):
                return True

        else:
            if abs(canvas_worst) > limit:
                return True

    return False



def F7_path_connectivity(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction | None = None,
    skeleton: Skeleton | None = None,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    """
    F7: Electrostatic path connectivity pruning.

    Return True  => discard layout
    Return False => keep layout

    This stricter version requires every input pin to be connected
    to the output region through the effective-radius connectivity graph.
    """
    if skeleton is None:
        return False
```

```
positions = layout.all_positions
n = len(positions)

if n == 0:
    return True

r_eff = params.connectivity_radius
if r_eff is None:
    r_eff = 3.0 * params.lambda_tf

output_keys = {coord_key(p) for p in skeleton.output_positions()}

output_indices = {
    i for i, p in enumerate(positions)
    if coord_key(p) in output_keys
}

if not output_indices:
    return True

adj = [[] for _ in range(n)]

for i in range(n):
    for j in range(i + 1, n):
        if euclidean(positions[i], positions[j]) <= r_eff:
            adj[i].append(j)
            adj[j].append(i)

def reaches_output(source_indices):
    if not source_indices:
        return False
```

```
    visited = set(source_indices)
    q = deque(source_indices)

    while q:
        i = q.popleft()

        if i in output_indices:
            return True

        for j in adj[i]:
            if j not in visited:
                visited.add(j)
                q.append(j)

    return False

# Every input pin must be connected to the output region.
for pin in skeleton.input_pins:
    pin_keys = {coord_key(pin.pos_a), coord_key(pin.pos_b)}

    source_indices = [
        i for i, p in enumerate(positions)
        if coord_key(p) in pin_keys
    ]

    if not reaches_output(source_indices):
        return True

return False
```

```
def F8_output_potential_bound(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction,
    skeleton: Skeleton,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    for x in func.all_inputs():
        y = func.eval(x)

        skel_config = _fixed_config_for_io(skeleton, x, y)
        skel_pos, skel_chg = _config_positions_charges(skel_config)

        output_config = skeleton.output_charge_config(y)

        for site_pos, required_q in output_config:
            base_v = _potential_from_sources(
                site_pos,
                skel_pos,
                skel_chg,
                params,
            )

            canvas_min = sum(
                -get_potential(site_pos, c, params)
                for c in layout.canvas_positions
            )

            canvas_max = 0.0
```

```
            v_min = base_v + canvas_min
            v_max = base_v + canvas_max

            if not interval_intersects_charge_state(
                v_min,
                v_max,
                required_q,
                params,
            ):
                return True

    return False


def F9_electrostatic_pressure(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction,
    skeleton: Skeleton,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    margin = params.pressure_margin

    for x in func.all_inputs():
        y = func.eval(x)

        skel_config = _fixed_config_for_io(skeleton, x, y)
        skel_pos, skel_chg = _config_positions_charges(skel_config)

        for out_idx, pin in enumerate(skeleton.output_pins):
```

```
required_bit = int(y[out_idx])


a = pin.pos_a

b = pin.pos_b


exclude = {coord_key(a), coord_key(b)}


v_a = _potential_from_sources(
    a,
    skel_pos,
    skel_chg,
    params,
    exclude_keys=exclude,
)


v_b = _potential_from_sources(
    b,
    skel_pos,
    skel_chg,
    params,
    exclude_keys=exclude,
)


delta_base = v_a - v_b


d_min = delta_base

d_max = delta_base


for c in layout.canvas_positions:
    delta_neg = (
        -get_potential(a, c, params)
```

```
                - (-get_potential(b, c, params))
            )

            d_min += min(0.0, delta_neg)
            d_max += max(0.0, delta_neg)

        if params.bit0_requires_positive_pressure:
            if required_bit == 0:
                if d_max <= margin:
                    return True
            else:
                if d_min >= -margin:
                    return True
        else:
            if required_bit == 0:
                if d_min >= -margin:
                    return True
            else:
                if d_max <= margin:
                    return True

    return False



def _energy_with_neutral_canvas(
    layout: Layout,
    fixed_config: List[Tuple[np.ndarray, int]],
    params: PhysicalParameters,
) -> float:
    pos, chg = _config_positions_charges(fixed_config)
```

```
    positions = pos + layout.canvas_positions
    charges = chg + [0] * layout.n_canvas

    return get_total_energy(positions, charges, params)



def _relaxed_min_energy_for_io(
    layout: Layout,
    fixed_config: List[Tuple[np.ndarray, int]],
    params: PhysicalParameters,
) -> float:
    """
    Layout-sensitive relaxed energy.

    Enumerates canvas charge states {-1, 0}^d and returns the minimum
    total energy without metastability / hop checks.

    This is cheaper than F11/F12 strict physical feasibility because it
    only evaluates energy values.
    """
    skel_pos, skel_chg = _config_positions_charges(fixed_config)

    positions = skel_pos + layout.canvas_positions

    best = float("inf")

    for canvas_charges in product([-1, 0], repeat=layout.n_canvas):
        charges = skel_chg + list(canvas_charges)
        e = get_total_energy(positions, charges, params)

        if e < best:
```

```
            best = e

    return float(best)



def F10_energy_lower_bound(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction,
    skeleton: Skeleton,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    """
    F10: Layout-sensitive relaxed energy guard.

    Return True  => discard layout
    Return False => keep layout

    This version compares the relaxed minimum energy of the correct
    output assignment against competing incorrect output assignments.

    It is intentionally a coarse pre-F12 guard:
        - F10 uses params.energy_bound_margin
        - F12 uses params.io_instability_margin

    Recommended:
        energy_bound_margin should usually be larger than io_instability_margin,
        so F10 removes only strong energetic failures and F12 remains the final
        finer energetic check.
    """
```

```
tol = params.energy_tolerance

if params.energy_bound_margin is None:
    # Conservative default. If this is too large, F10 may prune nothing.
    margin = abs(params.mu_minus) * max(1, layout.n_canvas)
else:
    margin = float(params.energy_bound_margin)

threshold = max(tol, margin)

for x in func.all_inputs():
    y_correct = func.eval(x)

    e_correct = _relaxed_min_energy_for_io(
        layout,
        _fixed_config_for_io(skeleton, x, y_correct),
        params,
    )

    if params.f10_check_input_inversions:
        input_patterns = list(func.all_inputs())
    else:
        input_patterns = [x]

    for x_alt in input_patterns:
        for y_alt in func.all_outputs():
            if x_alt == x and y_alt == y_correct:
                continue

            e_alt = _relaxed_min_energy_for_io(
                layout,
```

```
            _fixed_config_for_io(skeleton, x_alt, y_alt),
            params,
        )

        # If an incorrect assignment is lower by more than margin,
        # reject this layout.
        if e_alt < e_correct - threshold:
            return True

    return False



def ground_state_energy_for_io(
    layout: Layout,
    input_pattern: Tuple[int, ...],
    output_pattern: Tuple[int, ...],
    skeleton: Skeleton,
    params: PhysicalParameters,
) -> float:
    cache_key = (
        "ground_state_energy_for_io",
        tuple(input_pattern),
        tuple(output_pattern),
        params.cache_key(),
    )

    if cache_key in layout.cache:
        return layout.cache[cache_key]

    fixed_config = _fixed_config_for_io(
```

```
    skeleton,
    input_pattern,
    output_pattern,
)

skel_pos, skel_chg = _config_positions_charges(fixed_config)

positions = skel_pos + layout.canvas_positions
n_fixed = len(skel_pos)

free_indices = _canvas_free_indices(
    n_fixed,
    len(positions),
    params,
)

best_metastable_e = float("inf")
best_any_e = float("inf")

for canvas_charges in product([-1, 0], repeat=layout.n_canvas):
    charges = skel_chg + list(canvas_charges)

    e = get_total_energy(positions, charges, params)

    if e < best_any_e:
        best_any_e = e

    if is_metastable(
        positions,
        charges,
        params,
```

```
            free_indices=free_indices,
            check_hops=params.check_configuration_stability,
        ):
            if e < best_metastable_e:
                best_metastable_e = e

    if math.isfinite(best_metastable_e):
        result = best_metastable_e
    elif params.relaxed_enumeration:
        result = best_any_e
    else:
        result = float("inf")

    layout.cache[cache_key] = result
    return result


def F11_physical_infeasibility(
    layout: Layout,
    params: PhysicalParameters,
    func: BooleanFunction,
    skeleton: Skeleton,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    """
    F11: Physical infeasibility pruning.

    Return True  => discard layout
    Return False => keep layout
```

```
Default strict meaning:
    For every input x, at least one metastable canvas charge assignment
    must support the correct I/O state.

Optional robustness extension:
    params.f11_min_support_states = k

If k > 1, the layout must have at least k metastable supporting
canvas states for every input pattern.
"""
min_support = int(getattr(params, "f11_min_support_states", 1))
min_support = max(1, min_support)

for x in func.all_inputs():
    y = func.eval(x)

    fixed_config = _fixed_config_for_io(
        skeleton,
        x,
        y,
    )

    skel_pos, skel_chg = _config_positions_charges(fixed_config)

    positions = skel_pos + layout.canvas_positions

    n_fixed = len(skel_pos)

    free_indices = _canvas_free_indices(
        n_fixed,
        len(positions),
```

```
        params,
    )

    support_count = 0

    for canvas_charges in product([-1, 0], repeat=layout.n_canvas):
        charges = skel_chg + list(canvas_charges)

        ok = is_metastable(
            positions,
            charges,
            params,
            free_indices=free_indices,
            check_hops=params.check_configuration_stability,
        )

        if ok:
            support_count += 1

            if support_count >= min_support:
                break

    if support_count < min_support:
        return True

return False


def F12_io_signal_instability(
    layout: Layout,
    params: PhysicalParameters,
```

```
    func: BooleanFunction,
    skeleton: Skeleton,
    canvas: Canvas | None = None,
    **kwargs,
) -> bool:
    tol = max(params.energy_tolerance, params.io_instability_margin)

    for x in func.all_inputs():
        y_correct = func.eval(x)

        e_correct = ground_state_energy_for_io(
            layout,
            x,
            y_correct,
            skeleton,
            params,
        )

        if not math.isfinite(e_correct):
            return True

        if params.instability_check_inputs:
            input_patterns = list(func.all_inputs())
        else:
            input_patterns = [x]

        for x_alt in input_patterns:
            for y_alt in func.all_outputs():
                if x_alt == x and y_alt == y_correct:
                    continue
```

```
            e_alt = ground_state_energy_for_io(
                layout,
                x_alt,
                y_alt,
                skeleton,
                params,
            )

            if math.isfinite(e_alt) and e_alt < e_correct - tol:
                return True

    return False


ALL_QC12_FILTERS = [
    F1_min_distance,
    F2_symmetry,
    F3_wire_interference,
    F4_positive_charge,
    F5_charge_count_bound,
    F6_input_pin_disturbance,
    F7_path_connectivity,
    F8_output_potential_bound,
    F9_electrostatic_pressure,
    F10_energy_lower_bound,
    F11_physical_infeasibility,
    F12_io_signal_instability,
]

--------------------------------------------------------------------------------

# src/gates.py
```

```
from __future__ import annotations

from typing import Tuple, Dict, List

from .data_structures import BooleanFunction



class HexTruthTable:
    def __init__(self, num_inputs: int, hex_value: int):
        self.num_inputs = int(num_inputs)
        self.hex_value = int(hex_value)

    def __call__(self, x: Tuple[int, ...]) -> int:
        idx = 0

        for bit in x:
            idx = (idx << 1) | int(bit)

        return int((self.hex_value >> idx) & 1)



GATE_HEX_3IN: Dict[str, int] = {
    "AND3": 0x80,
    "XOR-AND": 0x28,
    "OR-AND": 0xA8,
    "ONEHOT": 0x16,
    "MAJ": 0xE8,
    "GAMBLE": 0x81,
    "DOT": 0x52,
    "ITE": 0xD8,
    "AND-XOR": 0x6A,
```

```
    "XOR3": 0x96,
}



def normalize_gate_name(name: str) -> str:
    return str(name).strip().upper()



def get_3input_gate(name: str) -> BooleanFunction:
    name = normalize_gate_name(name)

    if name not in GATE_HEX_3IN:
        raise ValueError(
            f"Unknown 3-input gate '{name}'. "
            f"Available gates: {list(GATE_HEX_3IN.keys())}"
        )

    return BooleanFunction(
        name=name,
        num_inputs=3,
        func=HexTruthTable(3, GATE_HEX_3IN[name]),
        num_outputs=1,
    )



def get_all_3input_gate_names() -> List[str]:
    return list(GATE_HEX_3IN.keys())
-------------------------------------------------------------------------------------------
# src/paper_benchmarks.py

from __future__ import annotations
```

```
from typing import List, Tuple

from .data_structures import Skeleton, Canvas



def paper_like_skeleton_3in_1out_25(bdl_spacing: float = 0.76) -> Skeleton:
    s = float(bdl_spacing)

    input_pins = [
        [(-25.00, 12.00), (-25.00 + s, 12.00)],
        [(-25.00, 0.00), (-25.00 + s, 0.00)],
        [(-25.00, -12.00), (-25.00 + s, -12.00)],
    ]

    output_pins = [
        [(10.00, 0.00), (10.00 + s, 0.00)],
    ]

    wire_coords = [
        (-20.0, 12.0),
        (-17.0, 10.0),
        (-14.0, 8.0),
        (-11.0, 6.0),
        (-8.0, 4.0),

        (-20.0, 0.0),
        (-17.0, 0.0),
        (-14.0, 0.0),
        (-11.0, 0.0),
        (-8.0, 0.0),
```

```
        (-20.0, -12.0),
        (-17.0, -10.0),
        (-14.0, -8.0),
        (-11.0, -6.0),
        (-8.0, -4.0),

        (-4.0, 0.0),
        (0.0, 0.0),
    ]

    return Skeleton(
        input_pins_coords=input_pins,
        output_pins_coords=output_pins,
        wire_coords=wire_coords,
    )


def paper_like_canvas_143() -> Canvas:
    positions: List[Tuple[float, float]] = []

    xs = [-8.5 + 3.0 * i for i in range(13)]
    ys = [-15.0 + 3.0 * j for j in range(11)]

    for x in xs:
        for y in ys:
            positions.append((float(x), float(y)))

    assert len(positions) == 143

    return Canvas(positions)
```

---

```
# src/physics.py


from __future__ import annotations


import math
from typing import List, Sequence


import numpy as np




def _lambertw_positive(z: float) -> float:
    z = float(z)


    if z == 0.0:
        return 0.0


    w = math.log1p(z) if z > 1.0 else z
    w = max(w, 1e-12)


    for _ in range(80):
        ew = math.exp(w)
        f = w * ew - z
        fp = ew * (w + 1.0)


        if abs(fp) < 1e-30:
            break


        w_new = w - f / fp


        if abs(w_new - w) < 1e-14:
```

```
            return float(w_new)

        w = max(w_new, 0.0)

    return float(w)



class PhysicalParameters:
    def __init__(
        self,
        mu_minus: float = -0.31,
        mu_plus: float = -0.80,
        lambda_tf: float = 5.0,
        epsilon_r: float = 5.6,
        qe: float = -1.0,
        k_coulomb: float = 1.43996,

        connectivity_radius: float | None = None,
        wire_forbidden_radius: float | None = None,

        charge_min_fraction: float = 0.0,
        charge_max_fraction: float = 1.0,

        input_disturbance_limit: float | None = None,
        enforce_input_population: bool = False,

        check_fixed_io_population: bool = False,
        check_configuration_stability: bool = True,

        relaxed_enumeration: bool = False,
```

```
    instability_check_inputs: bool = False,
    f10_check_input_inversions: bool = False,

    pressure_margin: float = 0.0,
    energy_tolerance: float = 1e-9,
    energy_bound_margin: float | None = None,

    io_instability_margin: float = 0.0,

    bit0_requires_positive_pressure: bool = False,
):
    self.mu_minus = float(mu_minus)
    self.mu_plus = float(mu_plus)
    self.lambda_tf = float(lambda_tf)
    self.epsilon_r = float(epsilon_r)
    self.qe = float(qe)
    self.k_coulomb = float(k_coulomb)

    self.connectivity_radius = connectivity_radius
    self.wire_forbidden_radius = wire_forbidden_radius

    self.charge_min_fraction = float(charge_min_fraction)
    self.charge_max_fraction = float(charge_max_fraction)

    self.input_disturbance_limit = input_disturbance_limit
    self.enforce_input_population = bool(enforce_input_population)

    self.check_fixed_io_population = bool(check_fixed_io_population)
    self.check_configuration_stability = bool(check_configuration_stability)
    self.relaxed_enumeration = bool(relaxed_enumeration)
```

```
    self.instability_check_inputs = bool(instability_check_inputs)
    self.f10_check_input_inversions = bool(f10_check_input_inversions)

    self.pressure_margin = float(pressure_margin)
    self.energy_tolerance = float(energy_tolerance)
    self.energy_bound_margin = energy_bound_margin
    self.io_instability_margin = float(io_instability_margin)

    self.bit0_requires_positive_pressure = bool(bit0_requires_positive_pressure)

    self.d_min = self._compute_d_min()

def _compute_d_min(self) -> float:
    if self.lambda_tf <= 0 or abs(self.mu_plus) < 1e-15:
        return 0.0

    A = self.k_coulomb / self.epsilon_r
    z = A / (self.lambda_tf * abs(self.mu_plus))

    return self.lambda_tf * _lambertw_positive(z)

def cache_key(self) -> tuple:
    return (
        round(self.mu_minus, 12),
        round(self.mu_plus, 12),
        round(self.lambda_tf, 12),
        round(self.epsilon_r, 12),
        round(self.qe, 12),
        round(self.k_coulomb, 12),
        self.check_fixed_io_population,
        self.check_configuration_stability,
```

```
            self.relaxed_enumeration,
            round(self.energy_tolerance, 15),
            round(self.io_instability_margin, 15),
        )



def euclidean(
    p1: Sequence[float] | np.ndarray,
    p2: Sequence[float] | np.ndarray,
) -> float:
    return float(
        np.linalg.norm(
            np.asarray(p1, dtype=float) - np.asarray(p2, dtype=float)
        )
    )



def get_potential(
    p1: Sequence[float] | np.ndarray,
    p2: Sequence[float] | np.ndarray,
    params: PhysicalParameters,
) -> float:
    d = euclidean(p1, p2)

    if d < 1e-12:
        return float("inf")

    return (
        params.k_coulomb
        / (params.epsilon_r * d)
        * math.exp(-d / params.lambda_tf)
```

```
    )



def get_local_potential(
    positions: List[np.ndarray],
    charges: List[int],
    i: int,
    params: PhysicalParameters,
) -> float:
    v = 0.0

    for j in range(len(positions)):
        if i == j:
            continue

        v += int(charges[j]) * get_potential(positions[i], positions[j], params)

    return float(v)



def get_total_energy(
    positions: List[np.ndarray],
    charges: List[int],
    params: PhysicalParameters,
) -> float:
    n = len(positions)
    e_interaction = 0.0

    for i in range(n):
        for j in range(i + 1, n):
            pot = get_potential(positions[i], positions[j], params)
```

```
            e_interaction += int(charges[i]) * int(charges[j]) * pot

    n_negative = sum(1 for q in charges if int(q) == -1)
    e_chemical = n_negative * params.mu_minus * abs(params.qe)

    return float(e_interaction + e_chemical)
--------------------------------------------------------------------------------
# src/reporting.py

from __future__ import annotations

import csv
import json
import os
from typing import Dict, Any, List, Optional



def ensure_dir(path: str) -> None:
    if path:
        os.makedirs(path, exist_ok=True)



def write_csv(path: str, rows: List[Dict[str, Any]], fieldnames: List[str]) -> None:
    ensure_dir(os.path.dirname(path))

    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()

        for row in rows:
            writer.writerow(row)
```

```
def write_json(path: str, data: Dict[str, Any]) -> None:
    ensure_dir(os.path.dirname(path))

    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2, ensure_ascii=False)



def write_markdown(path: str, content: str) -> None:
    ensure_dir(os.path.dirname(path))

    with open(path, "w", encoding="utf-8") as f:
        f.write(content)



def markdown_table(headers: List[str], rows: List[List[Any]]) -> str:
    out = []

    out.append("| " + " | ".join(headers) + " |")
    out.append("| " + " | ".join(["---" for _ in headers]) + " |")

    for row in rows:
        out.append("| " + " | ".join(str(x) for x in row) + " |")

    return "\n".join(out)



def safe_ratio(a: float, b: float) -> float:
    if b == 0:
        return float("inf")
```

```
    return float(a) / float(b)



def fmt_ratio(a: float, b: float) -> str:
    r = safe_ratio(a, b)

    if r == float("inf"):
        return "inf"

    return f"{r:.2f}×"



def percent(x: float) -> str:
    return f"{100.0 * x:.2f}%"



def write_xlsx_workbook(
    path: str,
    sheets: Dict[str, List[Dict[str, Any]]],
) -> Optional[str]:
    """
    Optional Excel workbook writer.

    Requires:
        pip install openpyxl

    If openpyxl is not installed, returns None and does not fail.
    """
    try:
        from openpyxl import Workbook
        from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
```

```
    from openpyxl.utils import get_column_letter
except Exception:
    return None

ensure_dir(os.path.dirname(path))

wb = Workbook()
default_ws = wb.active
wb.remove(default_ws)

header_fill = PatternFill("solid", fgColor="FFF2CC")
header_font = Font(bold=True)
thin = Side(border_style="thin", color="808080")
border = Border(left=thin, right=thin, top=thin, bottom=thin)

for sheet_name, rows in sheets.items():
    ws = wb.create_sheet(title=sheet_name[:31])

    if not rows:
        ws["A1"] = "No data"
        continue

    headers = list(rows[0].keys())

    for col_idx, h in enumerate(headers, start=1):
        cell = ws.cell(row=1, column=col_idx, value=h)
        cell.fill = header_fill
        cell.font = header_font
        cell.alignment = Alignment(horizontal="center")
        cell.border = border
```

```
        for row_idx, row in enumerate(rows, start=2):
            for col_idx, h in enumerate(headers, start=1):
                cell = ws.cell(row=row_idx, column=col_idx, value=row.get(h, ""))
                cell.border = border

        for col_idx, h in enumerate(headers, start=1):
            width = max(len(str(h)) + 2, 12)
            for row_idx in range(2, len(rows) + 2):
                value = ws.cell(row=row_idx, column=col_idx).value
                width = max(width, min(len(str(value)) + 2, 35))
            ws.column_dimensions[get_column_letter(col_idx)].width = width

        ws.freeze_panes = "A2"

    wb.save(path)
    return path
```

---

```
# src/siqad_export.py

from __future__ import annotations

import csv
import json
import os
from datetime import datetime
from html import escape
from typing import Dict, Any, List, Tuple

from .data_structures import Layout
```

```
DEFAULT_A1_X = 3.84
DEFAULT_A2_Y = 7.68

DEFAULT_BASIS = {
    0: (0.0, 0.0),
    1: (0.0, 2.25),
}



def _fmt(v: float) -> str:
    s = f"{float(v):.9f}"
    s = s.rstrip("0").rstrip(".")

    if s == "-0":
        s = "0"

    return s



def _arr_to_list(p):
    return [float(p[0]), float(p[1]), float(p[2])]



def _nearest_lattice_coord(
    x: float,
    y: float,
    used: set,
    a1_x: float = DEFAULT_A1_X,
    a2_y: float = DEFAULT_A2_Y,
    basis: Dict[int, Tuple[float, float]] = DEFAULT_BASIS,
    max_radius: int = 50,
```

```
) -> Dict[str, Any]:
    for radius in range(max_radius + 1):
        candidates = []

        for l, (bx, by) in basis.items():
            n0 = round((x - bx) / a1_x)
            m0 = round((y - by) / a2_y)

            for dn in range(-radius, radius + 1):
                for dm in range(-radius, radius + 1):
                    n = n0 + dn
                    m = m0 + dm

                    key = (int(n), int(m), int(l))

                    if key in used:
                        continue

                    sx = n * a1_x + bx
                    sy = m * a2_y + by

                    dist2 = (sx - x) ** 2 + (sy - y) ** 2
                    candidates.append((dist2, int(n), int(m), int(l), sx, sy))

        if candidates:
            candidates.sort(key=lambda t: t[0])
            dist2, n, m, l, sx, sy = candidates[0]
            used.add((n, m, l))

            return {
                "n": n,
```

```
            "m": m,
            "l": l,
            "siqad_x": float(sx),
            "siqad_y": float(sy),
            "snap_error": float(dist2 ** 0.5),
        }

    raise RuntimeError("Could not find an unused SiQAD lattice coordinate.")



def collect_layout_rows(layout: Layout) -> List[Dict[str, Any]]:
    rows: List[Dict[str, Any]] = []
    skeleton = layout.skeleton
    site_id = 0

    for pin_idx, pin in enumerate(skeleton.input_pins):
        rows.append({
            "site_id": site_id,
            "kind": "skeleton_input",
            "role": f"IN{pin_idx}_A",
            "pin_index": pin_idx,
            "x": float(pin.pos_a[0]),
            "y": float(pin.pos_a[1]),
            "z": float(pin.pos_a[2]),
        })
        site_id += 1

        rows.append({
            "site_id": site_id,
            "kind": "skeleton_input",
            "role": f"IN{pin_idx}_B",
```

```
        "pin_index": pin_idx,
        "x": float(pin.pos_b[0]),
        "y": float(pin.pos_b[1]),
        "z": float(pin.pos_b[2]),
    })
    site_id += 1

for pin_idx, pin in enumerate(skeleton.output_pins):
    rows.append({
        "site_id": site_id,
        "kind": "skeleton_output",
        "role": f"OUT{pin_idx}_A",
        "pin_index": pin_idx,
        "x": float(pin.pos_a[0]),
        "y": float(pin.pos_a[1]),
        "z": float(pin.pos_a[2]),
    })
    site_id += 1

    rows.append({
        "site_id": site_id,
        "kind": "skeleton_output",
        "role": f"OUT{pin_idx}_B",
        "pin_index": pin_idx,
        "x": float(pin.pos_b[0]),
        "y": float(pin.pos_b[1]),
        "z": float(pin.pos_b[2]),
    })
    site_id += 1

for wire_idx, w in enumerate(skeleton.wire_sidbs):
```

```
        p = w.coords

        rows.append({
            "site_id": site_id,
            "kind": "skeleton_wire",
            "role": f"WIRE{wire_idx}",
            "pin_index": "",
            "x": float(p[0]),
            "y": float(p[1]),
            "z": float(p[2]),
        })
        site_id += 1

    for canvas_idx, p in enumerate(layout.canvas_positions):
        rows.append({
            "site_id": site_id,
            "kind": "canvas",
            "role": f"CANVAS{canvas_idx}",
            "pin_index": "",
            "x": float(p[0]),
            "y": float(p[1]),
            "z": float(p[2]),
        })
        site_id += 1

    return rows



def prepare_siqad_rows(
    rows: List[Dict[str, Any]],
    snap_to_lattice: bool = True,
```

```
) -> List[Dict[str, Any]]:
    out = []
    used = set()

    for row in rows:
        r = dict(row)

        nearest = _nearest_lattice_coord(
            x=float(row["x"]),
            y=float(row["y"]),
            used=used,
        )

        r.update(nearest)

        if snap_to_lattice:
            r["export_x"] = nearest["siqad_x"]
            r["export_y"] = nearest["siqad_y"]
        else:
            r["export_x"] = float(row["x"])
            r["export_y"] = float(row["y"])

        out.append(r)

    return out



def color_for_kind(kind: str) -> str:
    if kind == "skeleton_input":
        return "#ff66ccff"
    if kind == "skeleton_output":
```

```
        return "#ffffcc66"
    if kind == "skeleton_wire":
        return "#ffc8c8c8"
    if kind == "canvas":
        return "#ff66dd66"
    return "#ffc8c8c8"



def write_rows_csv(path: str, rows: List[Dict[str, Any]]) -> None:
    os.makedirs(os.path.dirname(path), exist_ok=True)

    fieldnames = [
        "site_id",
        "kind",
        "role",
        "pin_index",
        "x",
        "y",
        "z",
        "n",
        "m",
        "l",
        "siqad_x",
        "siqad_y",
        "export_x",
        "export_y",
        "snap_error",
    ]

    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
```

```
        writer.writeheader()

        for row in rows:
            writer.writerow({k: row.get(k, "") for k in fieldnames})



def write_siqad_033_sqd(path: str, rows: List[Dict[str, Any]]) -> None:
    os.makedirs(os.path.dirname(path), exist_ok=True)

    if rows:
        xs = [float(r["export_x"]) for r in rows]
        ys = [float(r["export_y"]) for r in rows]
        margin = 10.0
        x1 = min(xs) - margin
        x2 = max(xs) + margin
        y1 = min(ys) - margin
        y2 = max(ys) + margin
    else:
        x1, x2, y1, y2 = -50, 50, -50, 50

    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    with open(path, "w", encoding="utf-8") as f:
        f.write('<?xml version="1.0" encoding="UTF-8"?>\n')
        f.write("<siqad>\n")

        f.write("    <program>\n")
        f.write("        <file_purpose>save</file_purpose>\n")
        f.write("        <version>0.3.3</version>\n")
        f.write(f"        <date>{escape(now)}</date>\n")
        f.write("    </program>\n")
```

```
f.write("    <gui>\n")
f.write("       <zoom>0.1</zoom>\n")
f.write(
   f'       <displayed_region x1="{_fmt(x1)}" y1="{_fmt(y1)}" '
   f'x2="{_fmt(x2)}" y2="{_fmt(y2)}"/>\n'
)
f.write('       <scroll x="0" y="0"/>\n')
f.write("    </gui>\n")

f.write("    <layers>\n")

f.write("       <layer_prop>\n")
f.write("          <name>Lattice</name>\n")
f.write("          <type>Lattice</type>\n")
f.write("          <role>Design</role>\n")
f.write("          <zoffset>0</zoffset>\n")
f.write("          <zheight>0</zheight>\n")
f.write("          <visible>1</visible>\n")
f.write("          <active>0</active>\n")
f.write("          <lat_vec>\n")
f.write('             <a1 x="3.84" y="0"/>\n')
f.write('             <a2 x="0" y="7.68"/>\n')
f.write("              <N>2</N>\n")
f.write('             <b1 x="0" y="0"/>\n')
f.write('             <b2 x="0" y="2.25"/>\n')
f.write("          </lat_vec>\n")
f.write("       </layer_prop>\n")

f.write("       <layer_prop>\n")
f.write("          <name>Screenshot Overlay</name>\n")
```

```
f.write("        <type>Misc</type>\n")
f.write("        <role>Overlay</role>\n")
f.write("        <zoffset>0</zoffset>\n")
f.write("        <zheight>0</zheight>\n")
f.write("        <visible>1</visible>\n")
f.write("        <active>0</active>\n")
f.write("      </layer_prop>\n")

f.write("      <layer_prop>\n")
f.write("        <name>Surface</name>\n")
f.write("        <type>DB</type>\n")
f.write("        <role>Design</role>\n")
f.write("        <zoffset>0</zoffset>\n")
f.write("        <zheight>0</zheight>\n")
f.write("        <visible>1</visible>\n")
f.write("        <active>0</active>\n")
f.write("      </layer_prop>\n")

f.write("      <layer_prop>\n")
f.write("        <name>Metal</name>\n")
f.write("        <type>Electrode</type>\n")
f.write("        <role>Design</role>\n")
f.write("        <zoffset>1000</zoffset>\n")
f.write("        <zheight>100</zheight>\n")
f.write("        <visible>1</visible>\n")
f.write("        <active>0</active>\n")
f.write("      </layer_prop>\n")

f.write("    </layers>\n")

f.write("    <design>\n")
```

```
    f.write('       <layer type="Lattice"/>\n')
    f.write('       <layer type="Misc"/>\n')
    f.write('       <layer type="DB">\n')

    for row in rows:
      color = color_for_kind(row["kind"])

      f.write(f"          <!-- {escape(str(row['kind']))} {escape(str(row['role']))} -->\n")
      f.write("          <dbdot>\n")
      f.write("             <layer_id>2</layer_id>\n")
      f.write(
        f'             <latcoord n="{int(row["n"])}" '
        f'm="{int(row["m"])}" l="{int(row["l"])}"/>\n'
      )
      f.write(
        f'             <physloc x="{_fmt(row["export_x"])}" '
        f'y="{_fmt(row["export_y"])}"/>\n'
      )
      f.write(f"             <color>{color}</color>\n")
      f.write("          </dbdot>\n")

    f.write("       </layer>\n")
    f.write('       <layer type="Electrode"/>\n')
    f.write("    </design>\n")
    f.write("</siqad>\n")


def _arr_to_list(p):
  return [float(p[0]), float(p[1]), float(p[2])]
```

```
def layout_to_json_dict(
    gate_name: str,
    candidate_id: int,
    layout: Layout,
    expected_truth_table: Dict[str, Any],
    siqad_rows: List[Dict[str, Any]],
) -> Dict[str, Any]:
    skeleton = layout.skeleton

    return {
        "gate": gate_name,
        "candidate_id": candidate_id,
        "canvas_indices": list(layout.canvas_indices),
        "input_pins": [
            {
                "pin_index": i,
                "pos_a": _arr_to_list(pin.pos_a),
                "pos_b": _arr_to_list(pin.pos_b),
                "encoding": {
                    "0": {"pos_a": -1, "pos_b": 0},
                    "1": {"pos_a": 0, "pos_b": -1},
                },
            }
            for i, pin in enumerate(skeleton.input_pins)
        ],
        "output_pins": [
            {
                "pin_index": i,
                "pos_a": _arr_to_list(pin.pos_a),
                "pos_b": _arr_to_list(pin.pos_b),
                "encoding": {
```

```
                "0": {"pos_a": -1, "pos_b": 0},
                "1": {"pos_a": 0, "pos_b": -1},
            },
        }
        for i, pin in enumerate(skeleton.output_pins)
    ],
    "wire_sidbs": [_arr_to_list(w.coords) for w in skeleton.wire_sidbs],
    "canvas_sidbs": [_arr_to_list(p) for p in layout.canvas_positions],
    "expected_truth_table": expected_truth_table,
    "siqad_export_rows": siqad_rows,
}


def write_candidate_export(
    root_dir: str,
    gate_name: str,
    candidate_id: int,
    layout: Layout,
    expected_truth_table: Dict[str, Any],
    snap_to_lattice: bool = True,
) -> Dict[str, str]:
    candidate_dir = os.path.join(
        root_dir,
        "candidates",
        f"candidate_{candidate_id:06d}",
    )

    os.makedirs(candidate_dir, exist_ok=True)

    raw_rows = collect_layout_rows(layout)
```

```
    siqad_rows = prepare_siqad_rows(
        raw_rows,
        snap_to_lattice=snap_to_lattice,
    )

    csv_path = os.path.join(candidate_dir, "all_sidbs.csv")
    sqd_path = os.path.join(candidate_dir, "candidate.sqd")
    json_path = os.path.join(candidate_dir, "candidate.json")

    write_rows_csv(csv_path, siqad_rows)
    write_siqad_033_sqd(sqd_path, siqad_rows)

    data = layout_to_json_dict(
        gate_name=gate_name,
        candidate_id=candidate_id,
        layout=layout,
        expected_truth_table=expected_truth_table,
        siqad_rows=siqad_rows,
    )

    with open(json_path, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2, ensure_ascii=False)

    return {
        "candidate_dir": candidate_dir,
        "csv": csv_path,
        "sqd": sqd_path,
        "json": json_path,
    }
-----------------------------------------------------------------
# src/stability.py
```

```
from __future__ import annotations

from typing import Iterable, List, Optional, Tuple

import numpy as np

from .physics import (
    PhysicalParameters,
    get_local_potential,
    get_total_energy,
)


def transition_threshold(mu: float, params: PhysicalParameters) -> float:
    return -float(mu) / params.qe


def negative_threshold(params: PhysicalParameters) -> float:
    return transition_threshold(params.mu_minus, params)


def positive_threshold(params: PhysicalParameters) -> float:
    return transition_threshold(params.mu_plus, params)


def neutral_window(params: PhysicalParameters) -> Tuple[float, float]:
    t1 = negative_threshold(params)
    t2 = positive_threshold(params)
    return min(t1, t2), max(t1, t2)
```

```
def is_population_stable_charge(
    charge: int,
    v_local: float,
    params: PhysicalParameters,
    tol: float = 1e-12,
) -> bool:
    charge = int(charge)

    neg_condition = params.mu_minus + params.qe * v_local < -tol
    pos_condition = params.mu_plus + params.qe * v_local > tol

    if charge == -1:
        return bool(neg_condition)

    if charge == 0:
        return bool((not neg_condition) and (not pos_condition))

    if charge == 1:
        return bool(pos_condition)

    return False



def charge_stability_window(
    charge: int,
    params: PhysicalParameters,
) -> Tuple[float, float]:
    charge = int(charge)

    t_minus = negative_threshold(params)
```

```
    t_plus = positive_threshold(params)

    lo = min(t_minus, t_plus)
    hi = max(t_minus, t_plus)

    if charge == -1:
        if params.qe < 0:
            return t_minus, float("inf")
        return float("-inf"), t_minus

    if charge == 0:
        return lo, hi

    if charge == 1:
        if params.qe < 0:
            return float("-inf"), t_plus
        return t_plus, float("inf")

    return float("inf"), float("-inf")



def interval_intersects_charge_state(
    v_min: float,
    v_max: float,
    charge: int,
    params: PhysicalParameters,
) -> bool:
    w_min, w_max = charge_stability_window(charge, params)
    return not (v_max < w_min or v_min > w_max)
```

```
def _local_potentials(
    positions: List[np.ndarray],
    charges: List[int],
    params: PhysicalParameters,
) -> List[float]:
    return [
        get_local_potential(positions, charges, i, params)
        for i in range(len(positions))
    ]



def is_metastable(
    positions: List[np.ndarray],
    charges: List[int],
    params: PhysicalParameters,
    free_indices: Optional[Iterable[int]] = None,
    check_hops: bool = True,
) -> bool:
    if len(positions) != len(charges):
        raise ValueError("positions and charges must have equal length.")

    if free_indices is None:
        free = list(range(len(positions)))
    else:
        free = sorted(set(int(i) for i in free_indices))

    local_vs = _local_potentials(positions, charges, params)

    for i in free:
        q = int(charges[i])
```

```
    if q not in (-1, 0, 1):
        return False

    if not is_population_stable_charge(q, local_vs[i], params):
        return False

if check_hops:
    base_e = get_total_energy(positions, charges, params)
    tol = params.energy_tolerance

    for i in free:
        if int(charges[i]) != -1:
            continue

        for j in free:
            if i == j:
                continue

            if int(charges[j]) != 0:
                continue

            new_charges = list(charges)
            new_charges[i] = 0
            new_charges[j] = -1

            new_e = get_total_energy(positions, new_charges, params)

            if new_e < base_e - tol:
                return False

return True
```